# Nawatech

Beyond Engineering

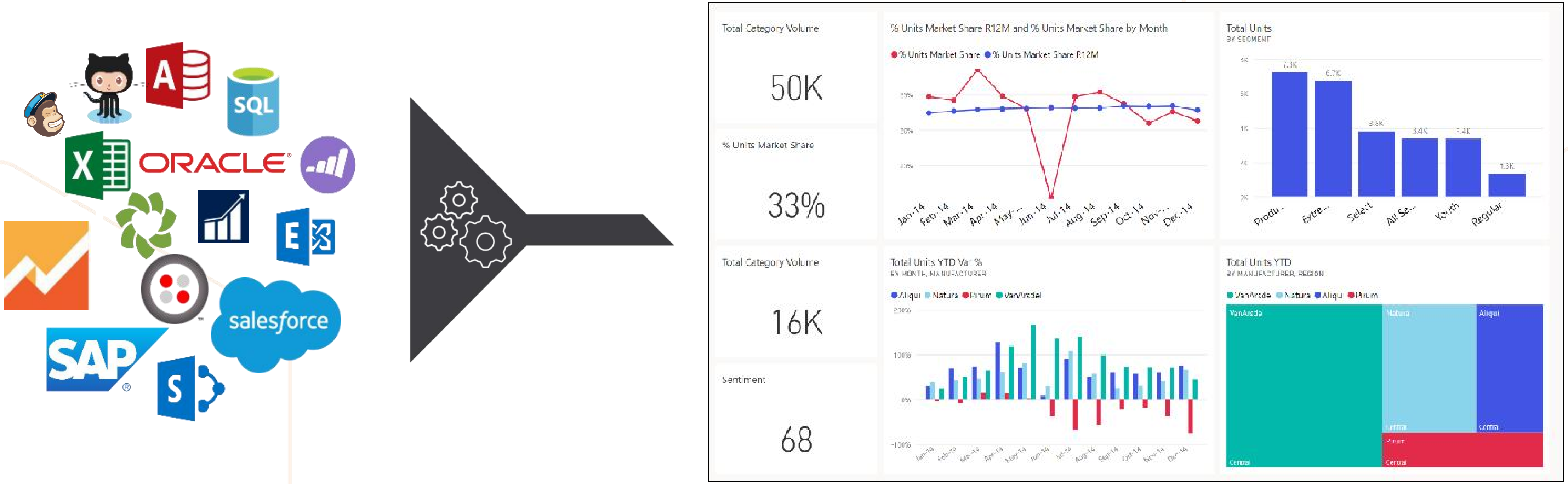
nawatech.co

# KAI Workshop– Day 5

Prepared for: PT Kereta Api Indonesia

# Discover data analysis

# From data to business insights with Power BI



# From data to business insights with Power BI

Microsoft | Customer Stories   All stories   Stories by product   Contact Sales   All Microsoft   Search

September 17, 2019

## Dutch Railways transforms transport with data-driven innovations

Share the story

## A new culture of innovation

With all of the customer data centrally accessible and able to be visualized using the Microsoft Power BI suite of business analytics tools, the team at Dutch Railways is now innovating like never before, coming up with new ideas for how to improve its service for customers.

"By using tools such as Microsoft Power BI, suddenly we are able to be much more agile as an organization, bringing together all sorts of different elements, insights, and data to drive innovations," says Geert. "It has shifted the attitude we have toward the way we work."

He has established an internal innovation community to bring people together from different units of the organization to discuss and develop ideas around themes like artificial intelligence, blockchain, and security.

"We start with a proof of concept and see if we can develop it into something scalable," say Geert. "That way we can see if an idea is worth investing more money into."

## An extra 20,000 seats every day

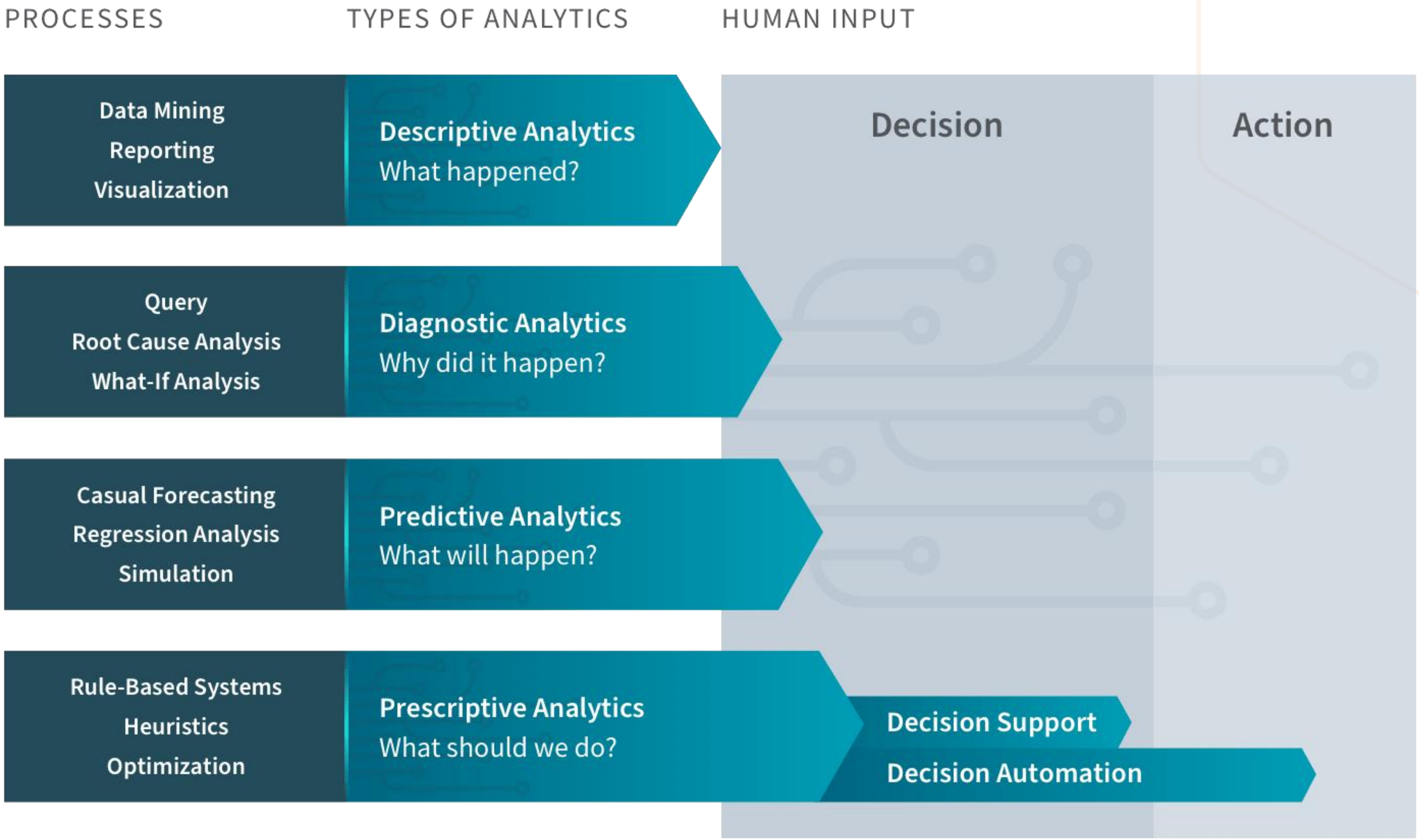
One such innovation was the Seat Finder app, which uses color coding to show passengers where to stand on the platform so they can find less crowded carriages when boarding.

## Different types of analysis

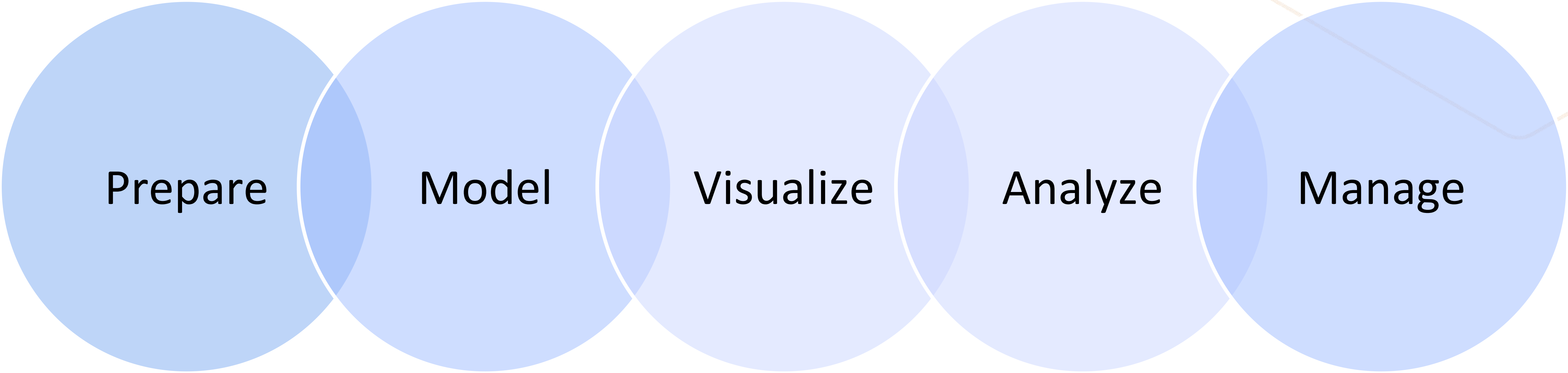
Data analysis is another form of storytelling with five categories:

- **Descriptive** : Summarize past data
- **Diagnostic** : Explain past data
- **Predictive** : Forecast future data
- **Prescriptive** : Optimize future data
- **Cognitive** : Learn from data

# Different types of analysis



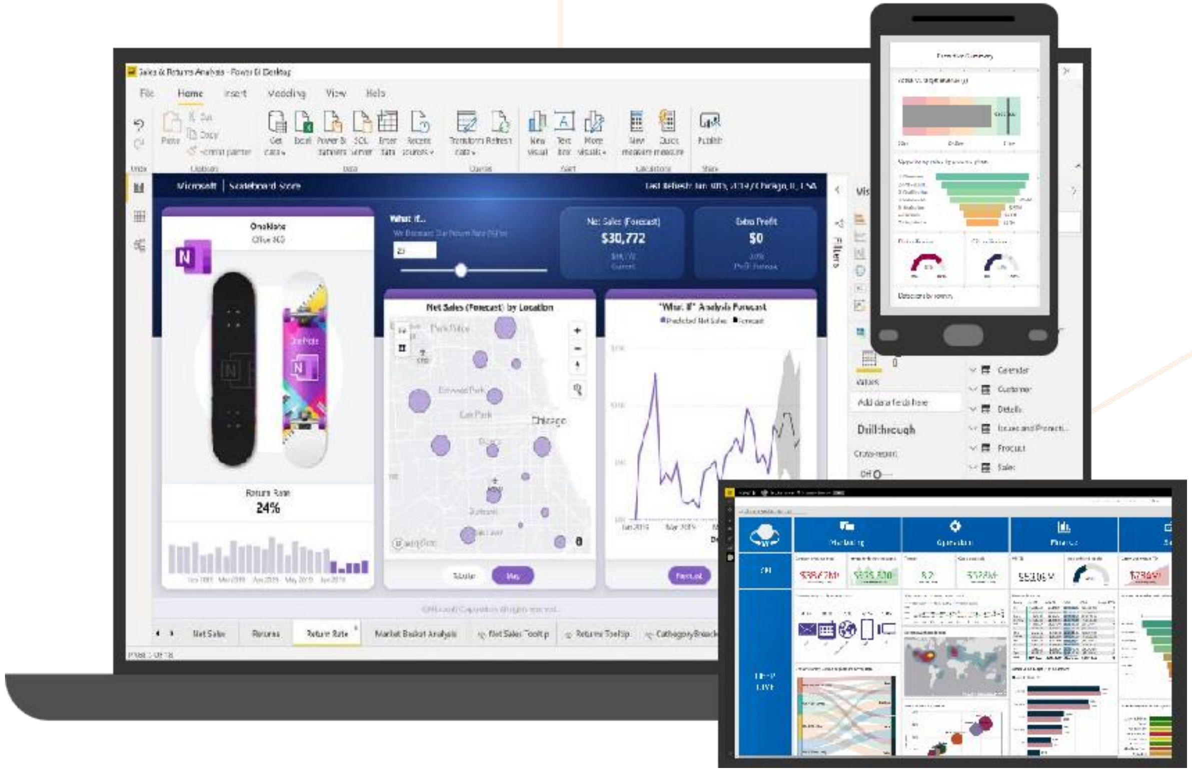
# Tasks of a Data Analyst



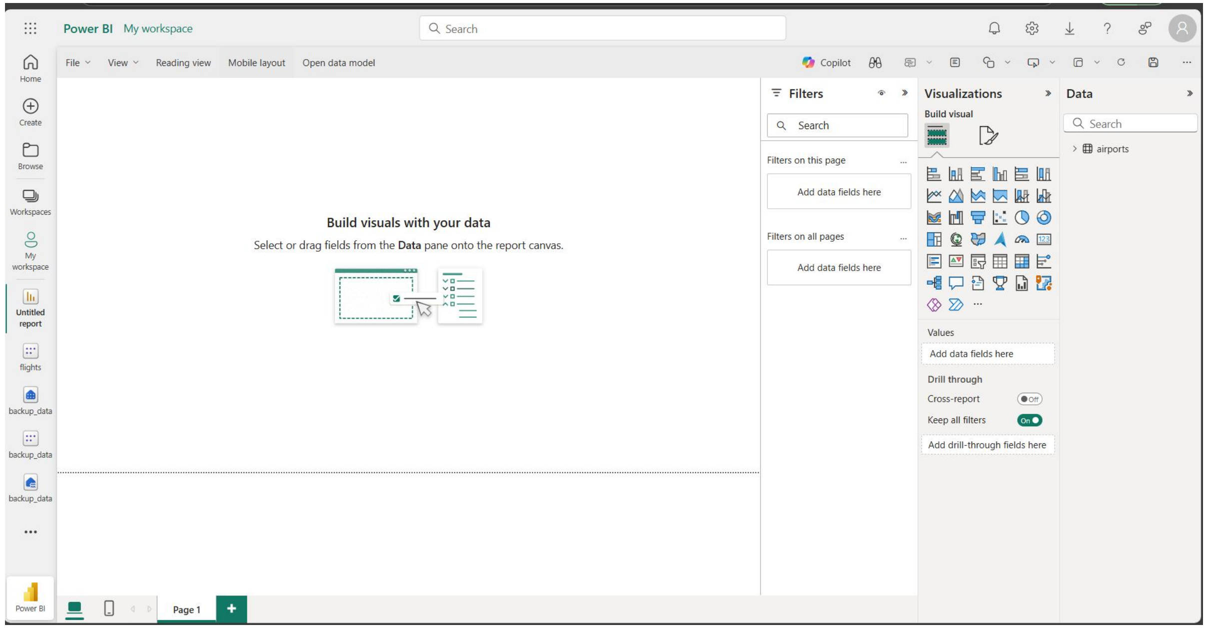
# Get started building with Power BI

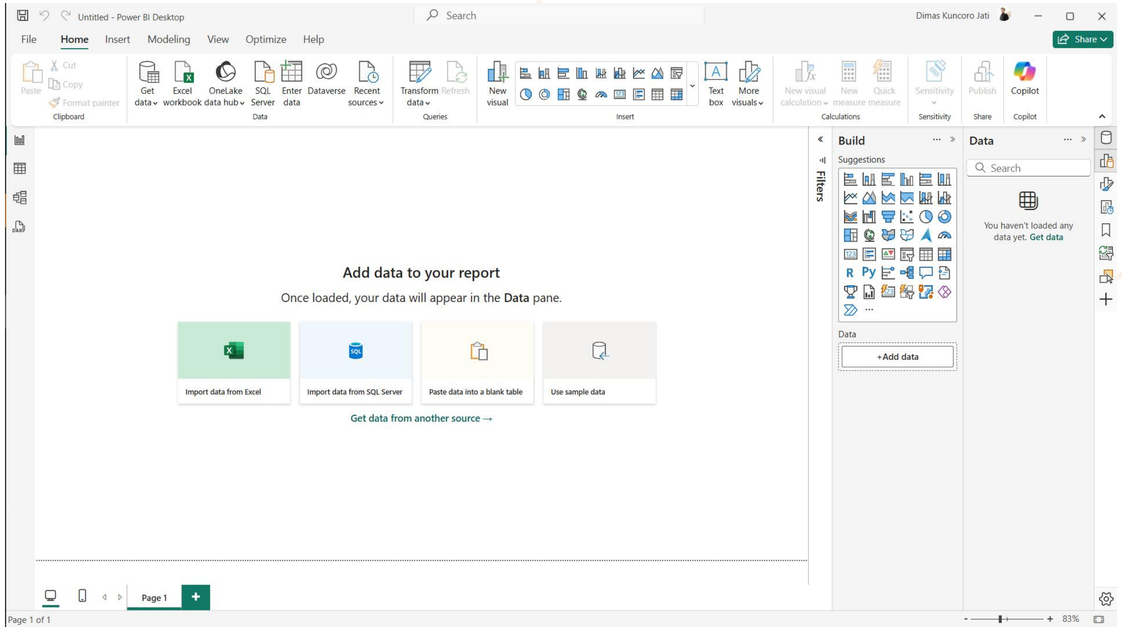
# Introduction to Power BI environment

- Create with Power BI Desktop
- Distribute with Power BI service
- Access with Power BI Mobile



# Power BI

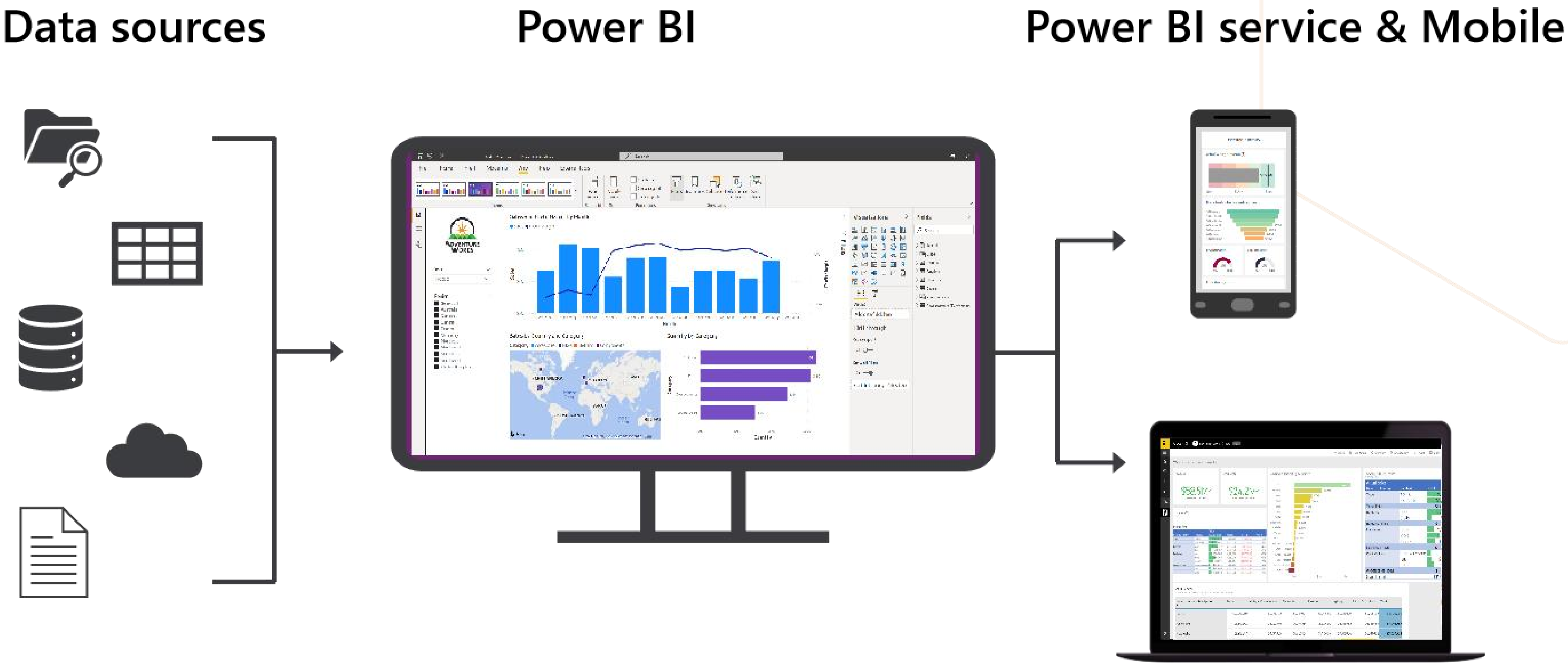



**Fabric**

**Power BI Desktop**

# The flow of Power BI



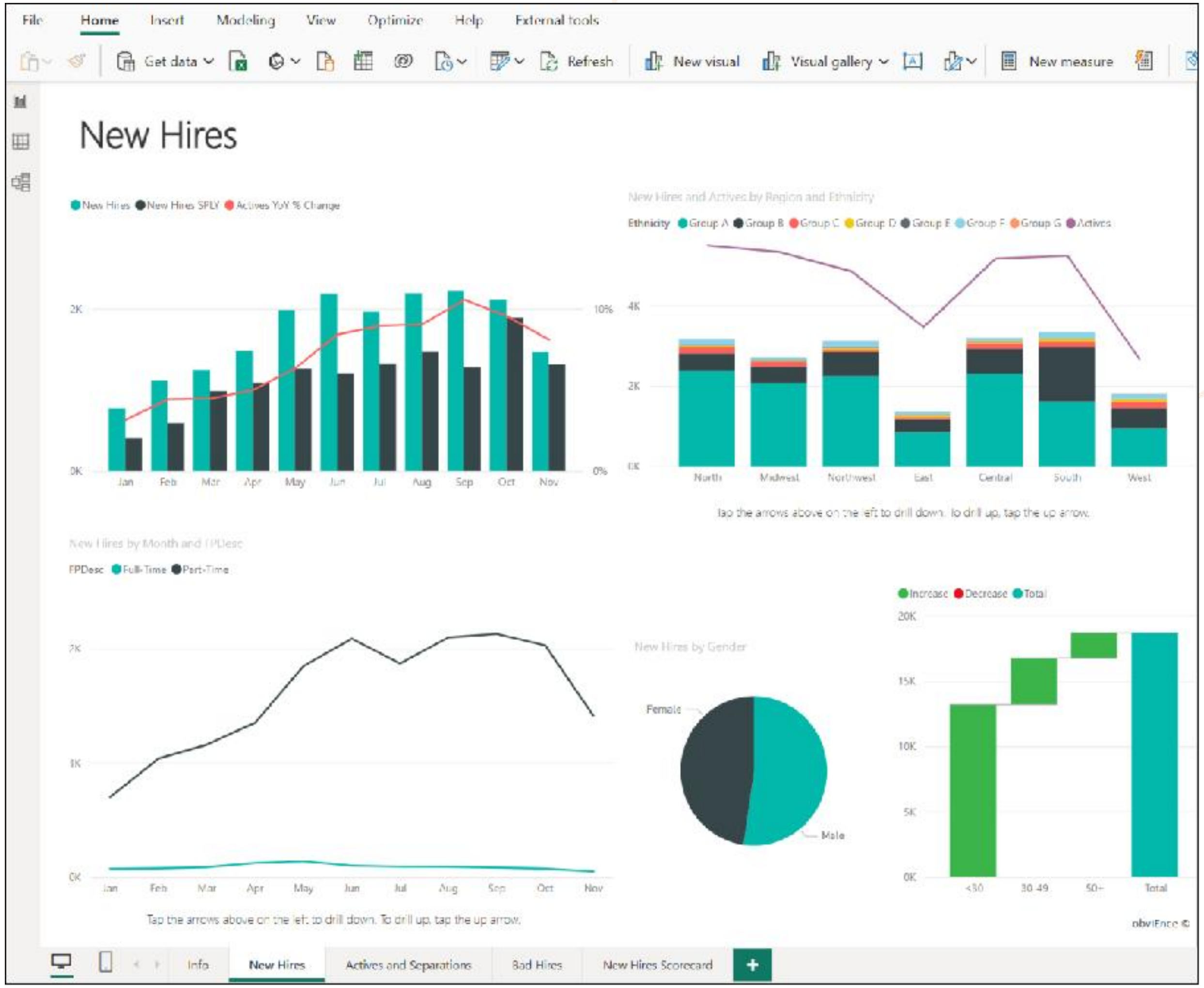
# The building blocks of Power BI

Semantic models

- Connected data, transformations, relationships, and calculations.

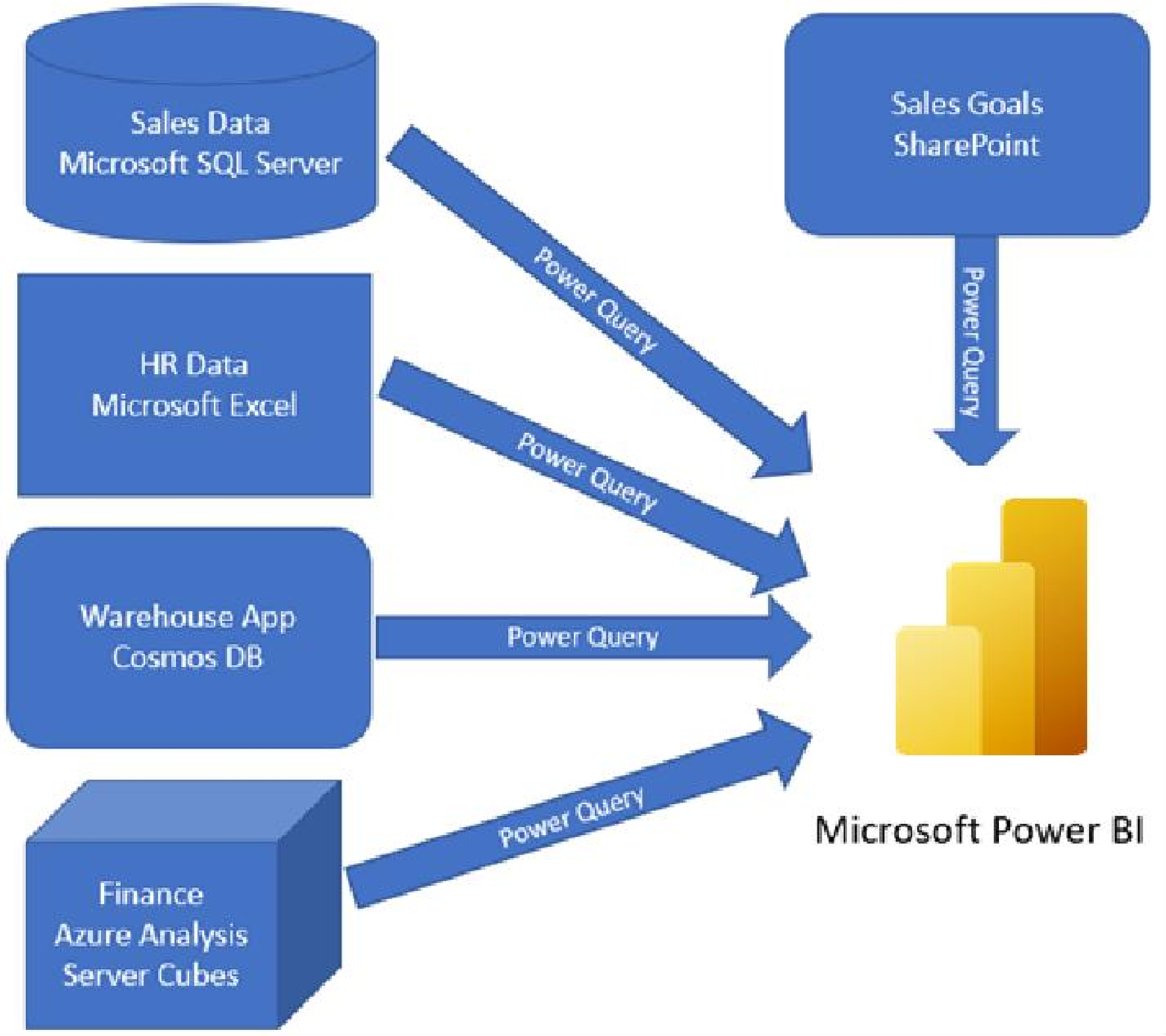
Visualizations

- Interactive representation of data through various charts and graphics.

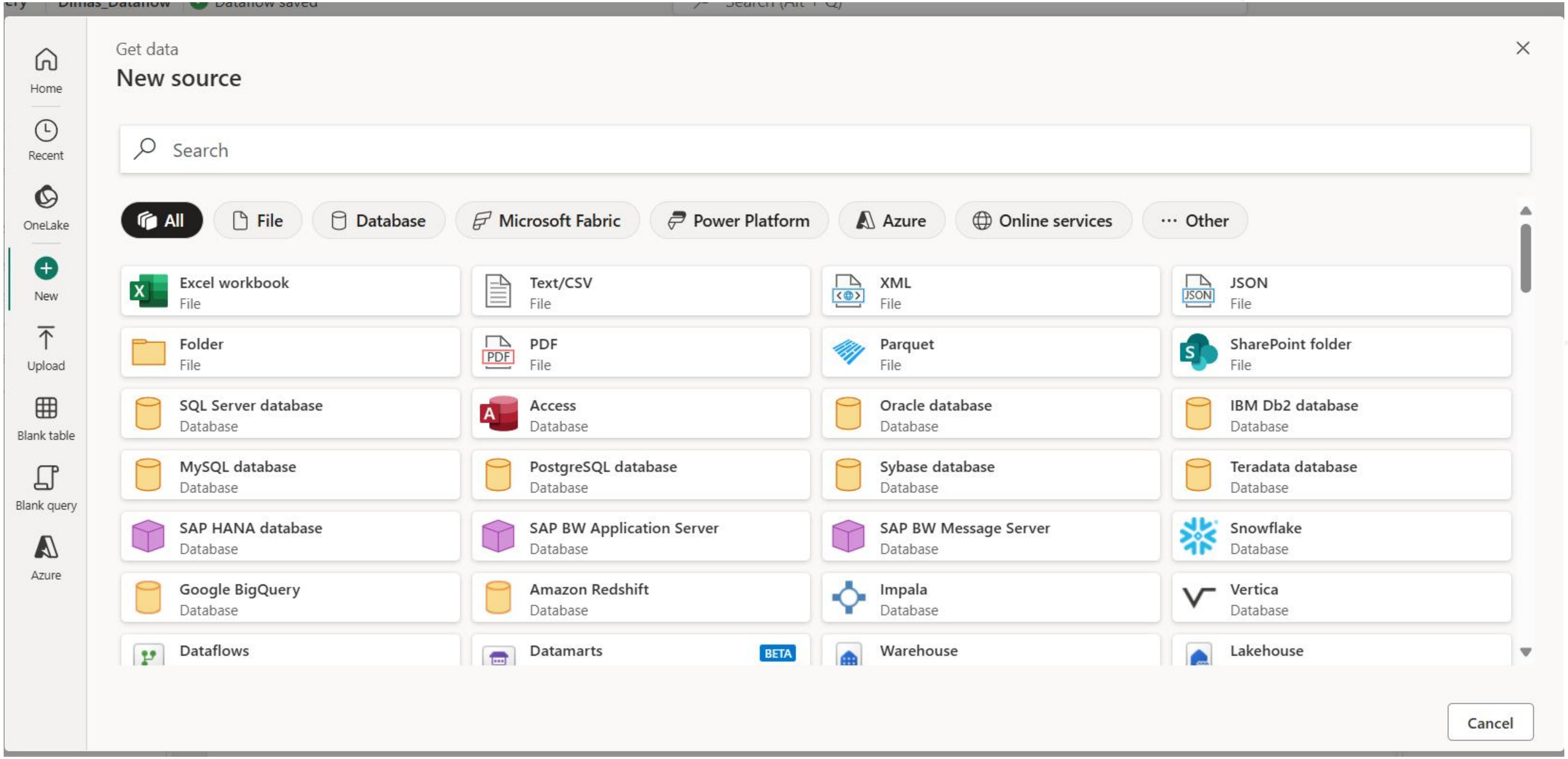


# Get data in Power BI

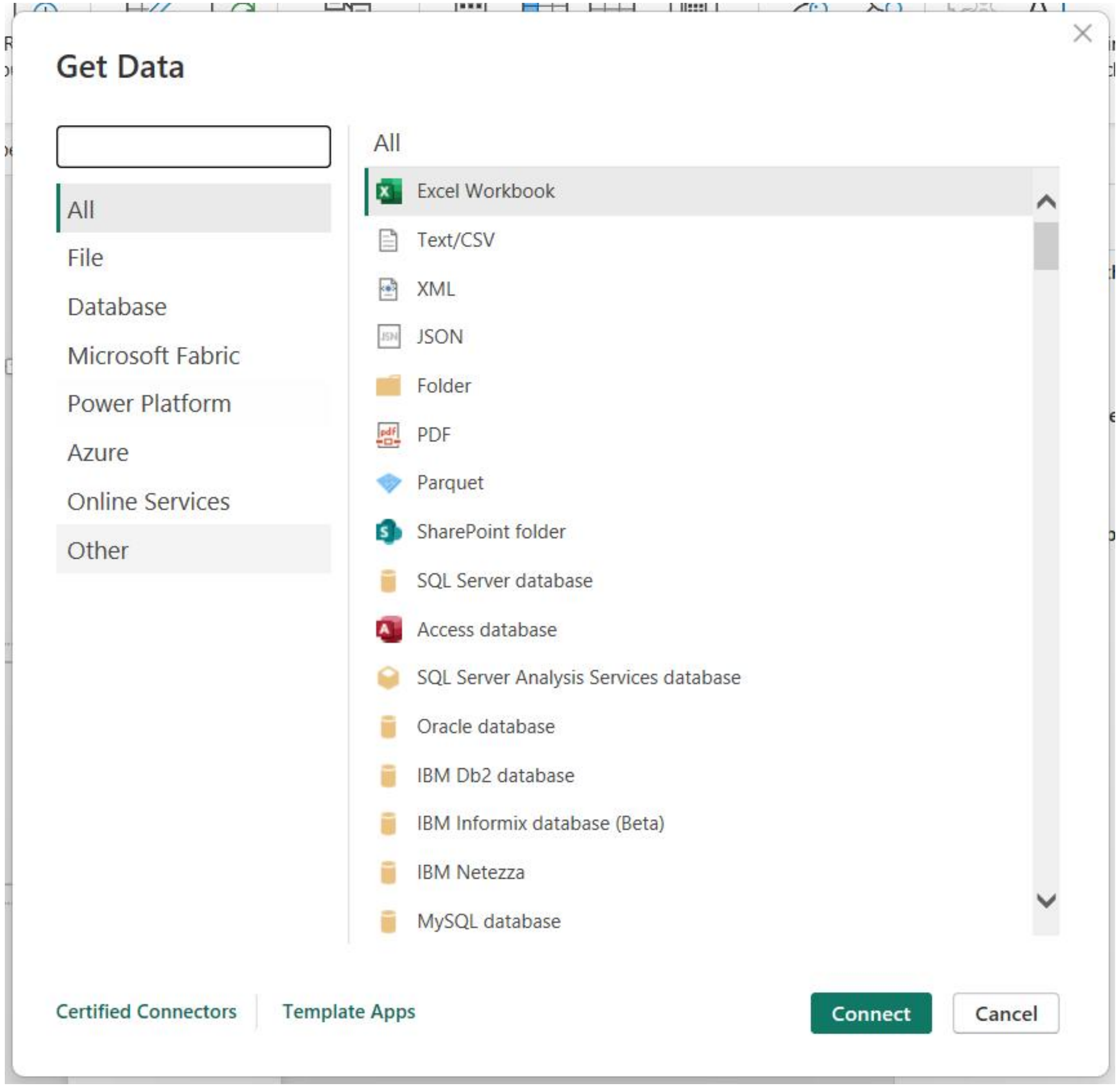
# Combine all data into a single semantic model



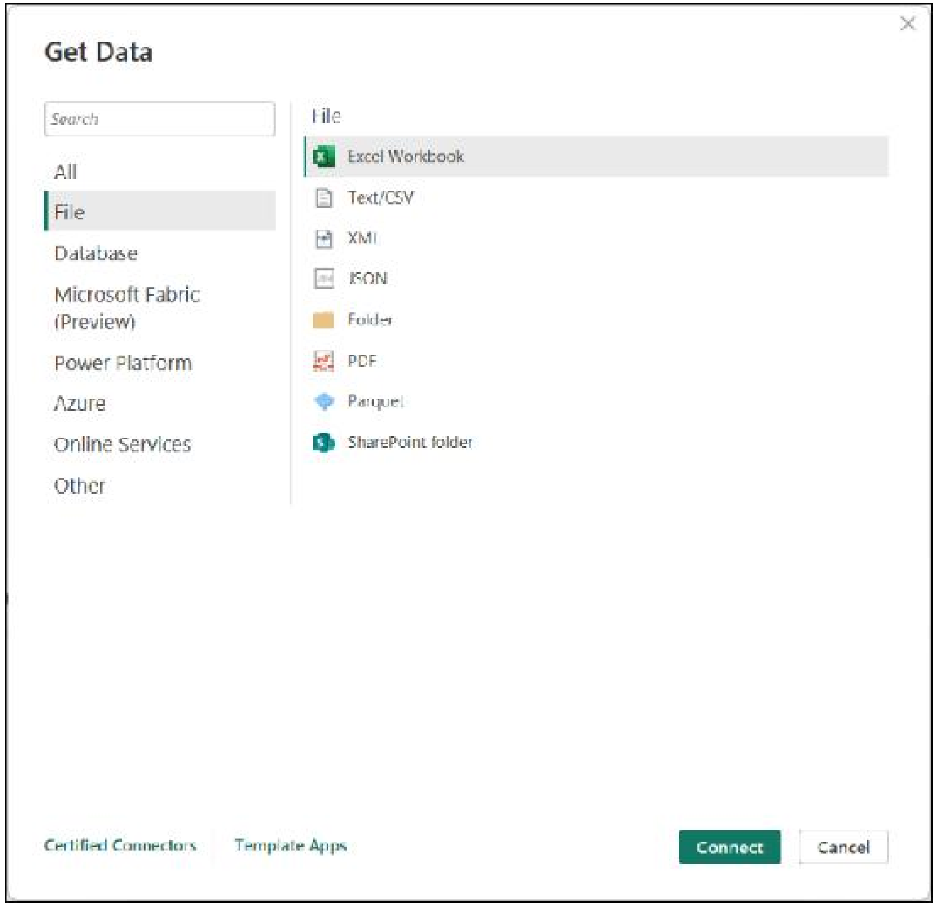
# Combine all data into a single semantic model in fabric

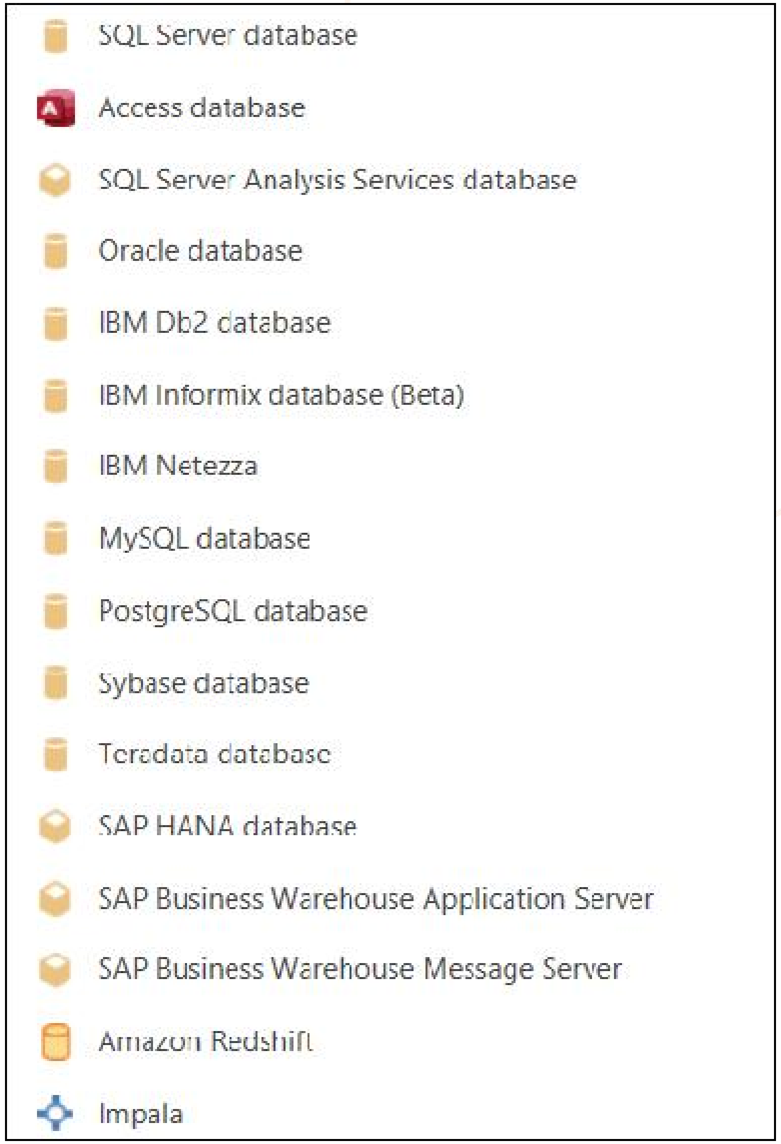


# Combine all data into a single semantic model in fabric



# Get data from a wide variety of sources



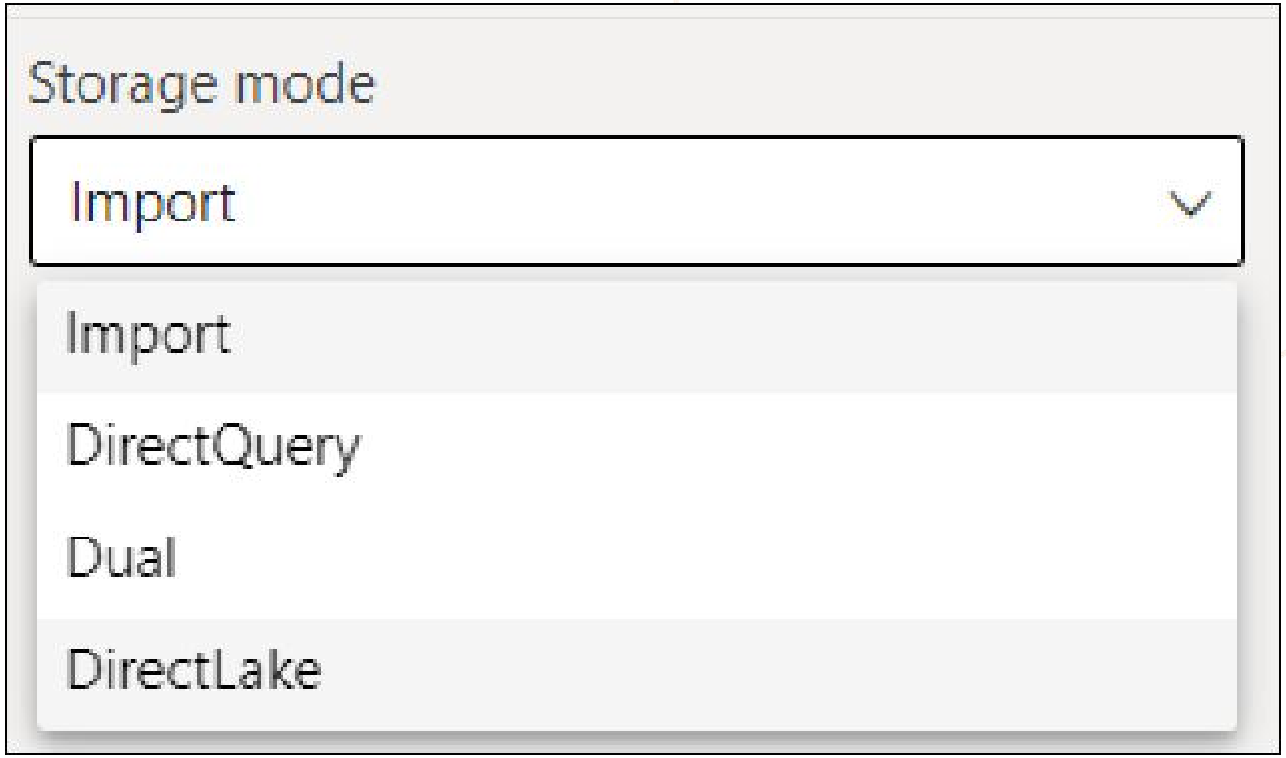



# Select a storage mode

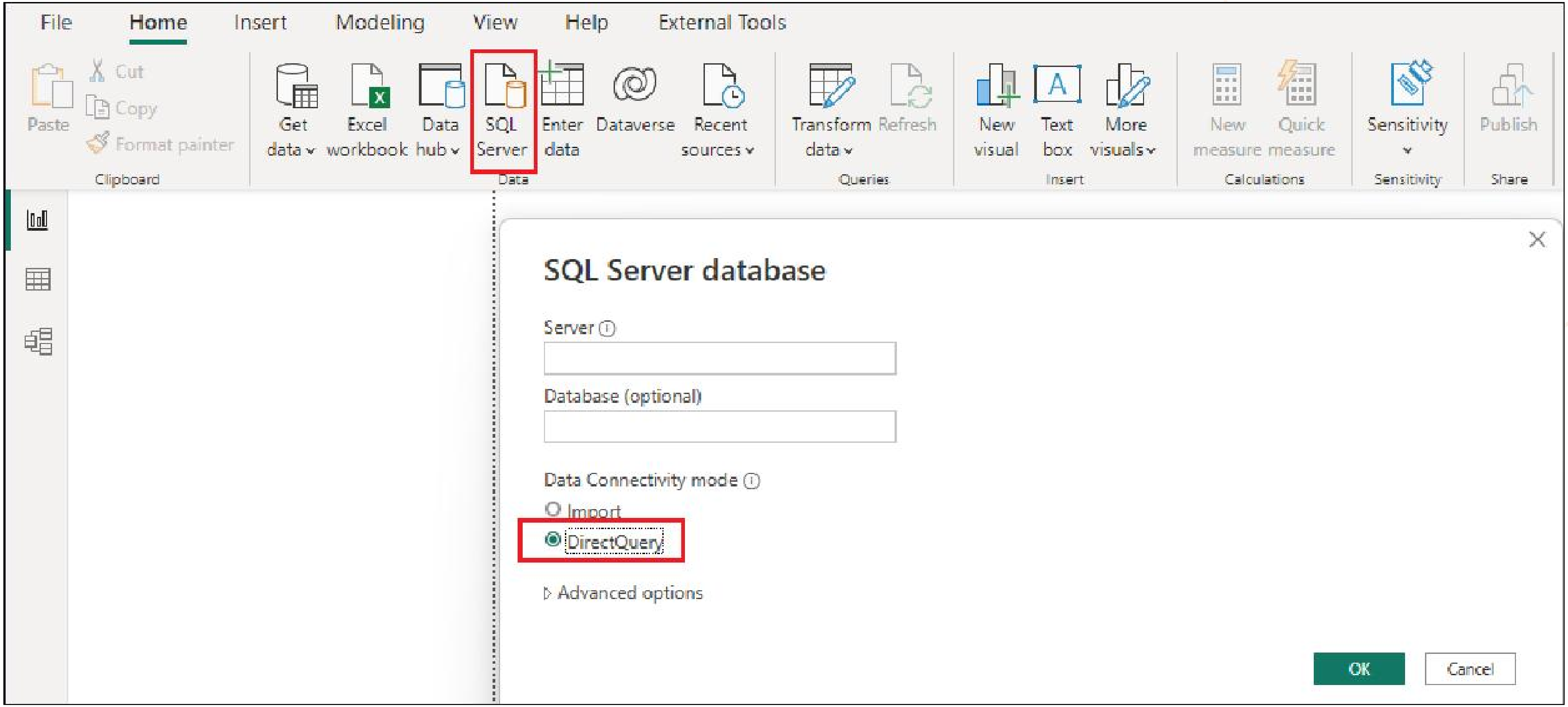
Storage mode affects

- Available transformations
- Report performance

*Not all sources support all modes



# Introduction to DirectQuery



# Implications of using DirectQuery

## Benefits

- Frequently changing data
- Need near real-time
- Large data volumes
- Multi-dimensional data

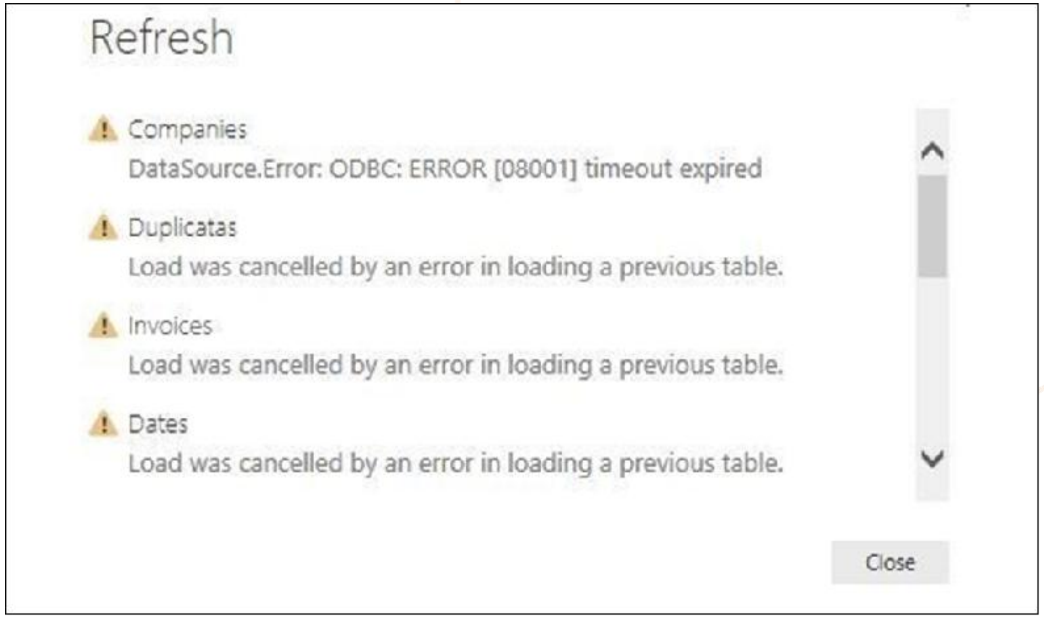
## Limitations

- Dependent on data source performance
- Security between source and destination
- Limited modeling capabilities
- Limited transformation features

# Data import errors

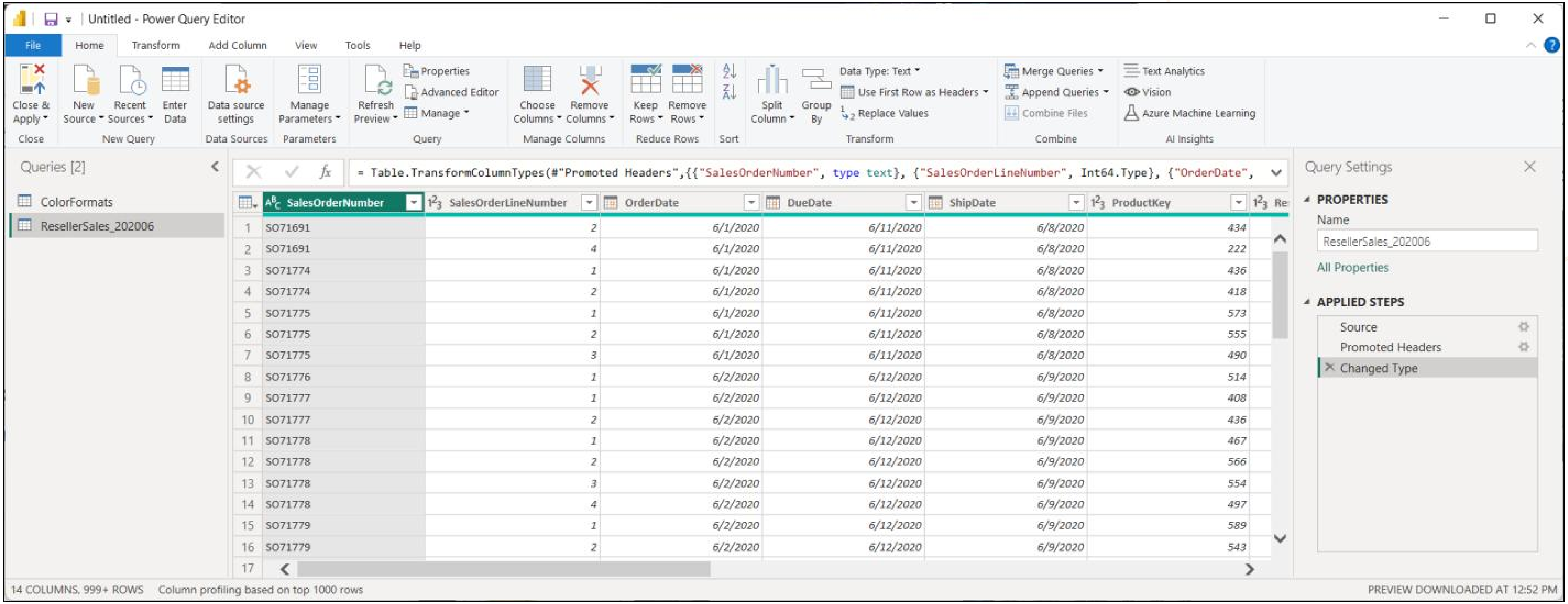
Possible data load errors:

- Query Timeout
- Couldn’t find data formatted as a table
- Could not find file
- Data type errors



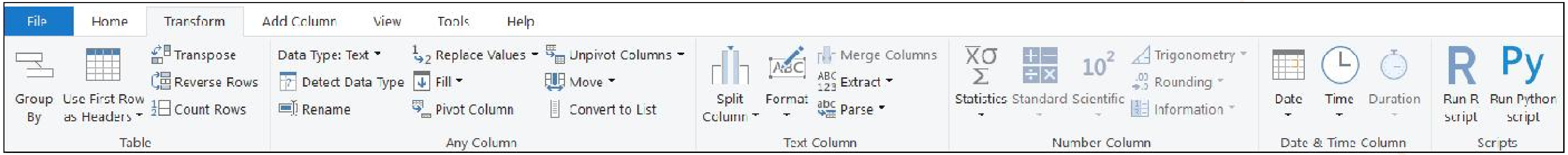
# Clean, transform, and load data

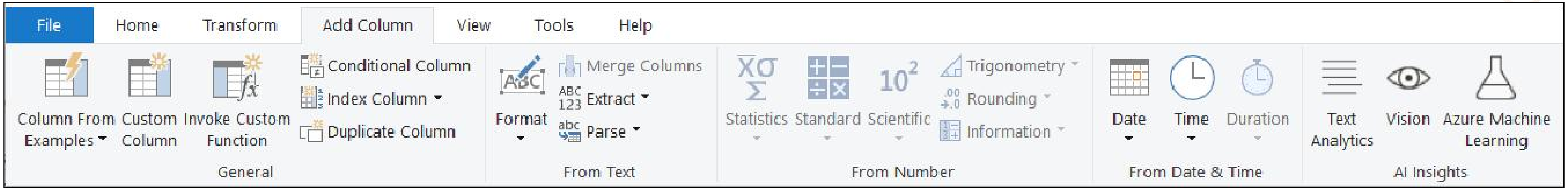
# Transform data with Power Query Editor



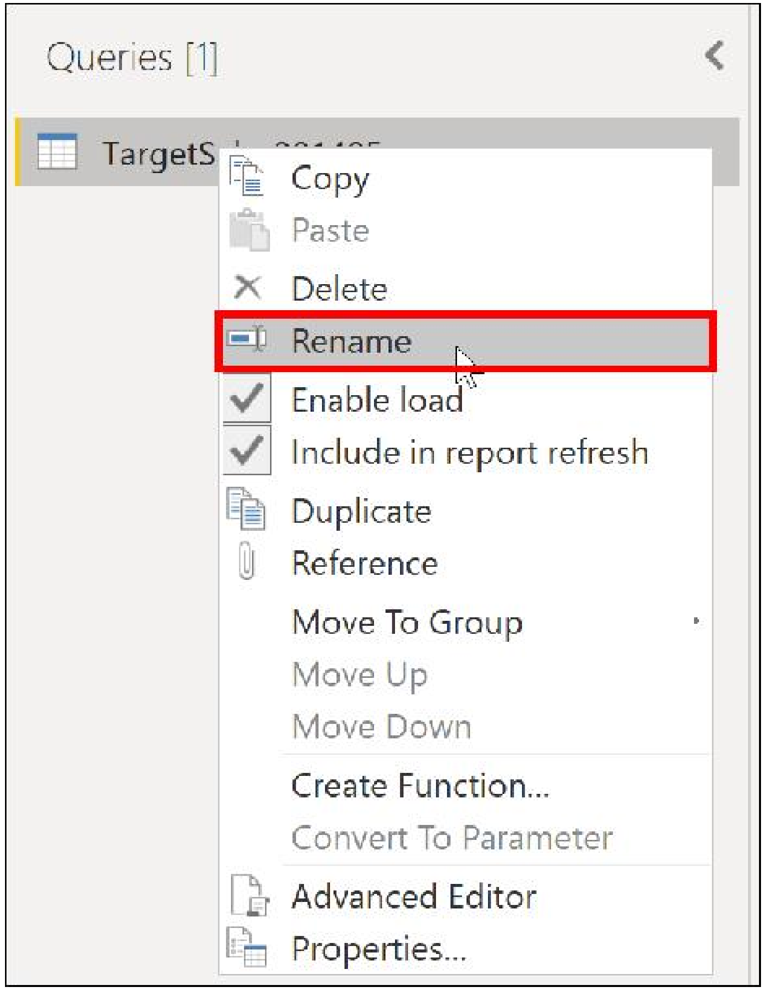
# Common transformations

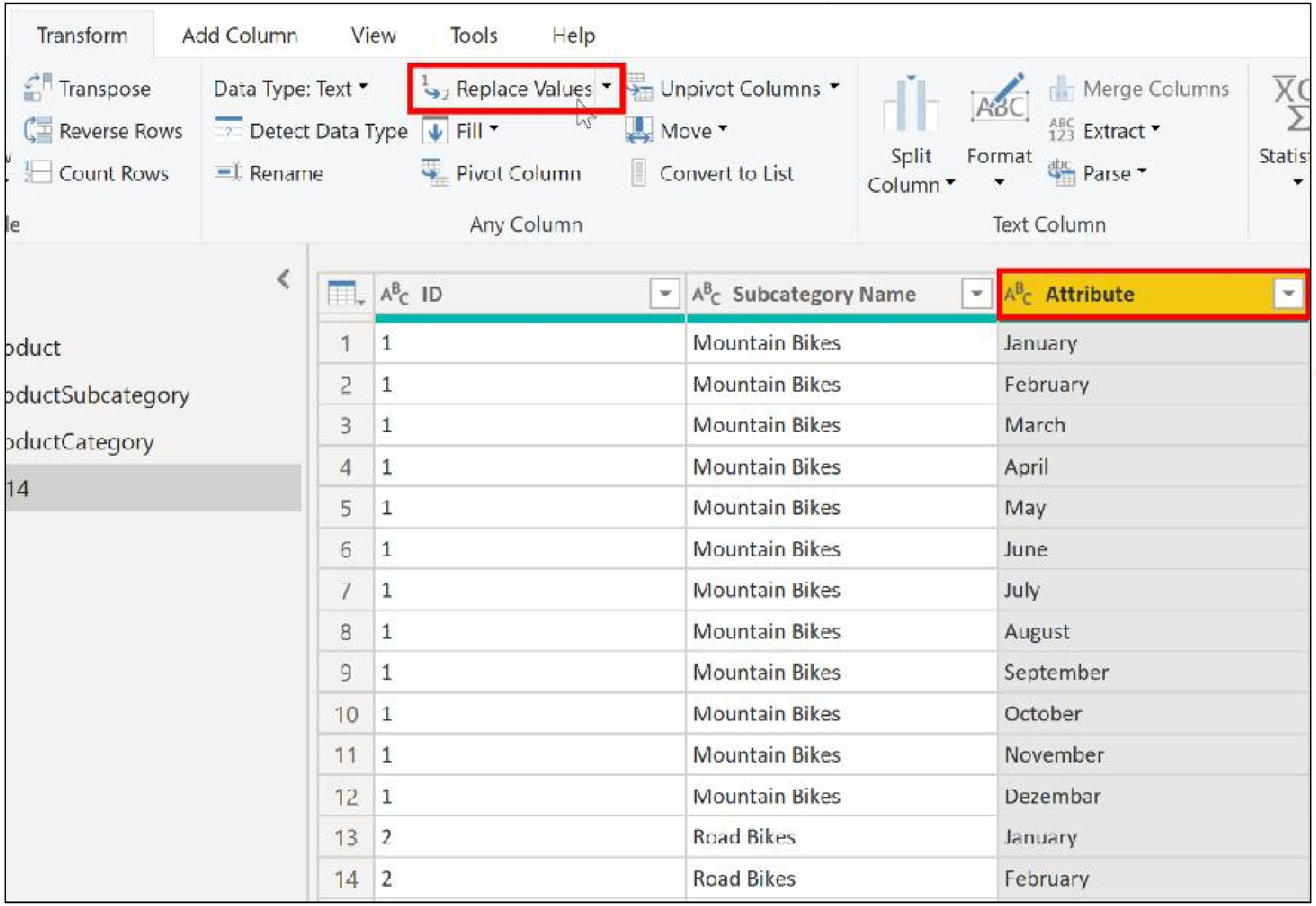
- Transform columns, add new, split, extract, and more



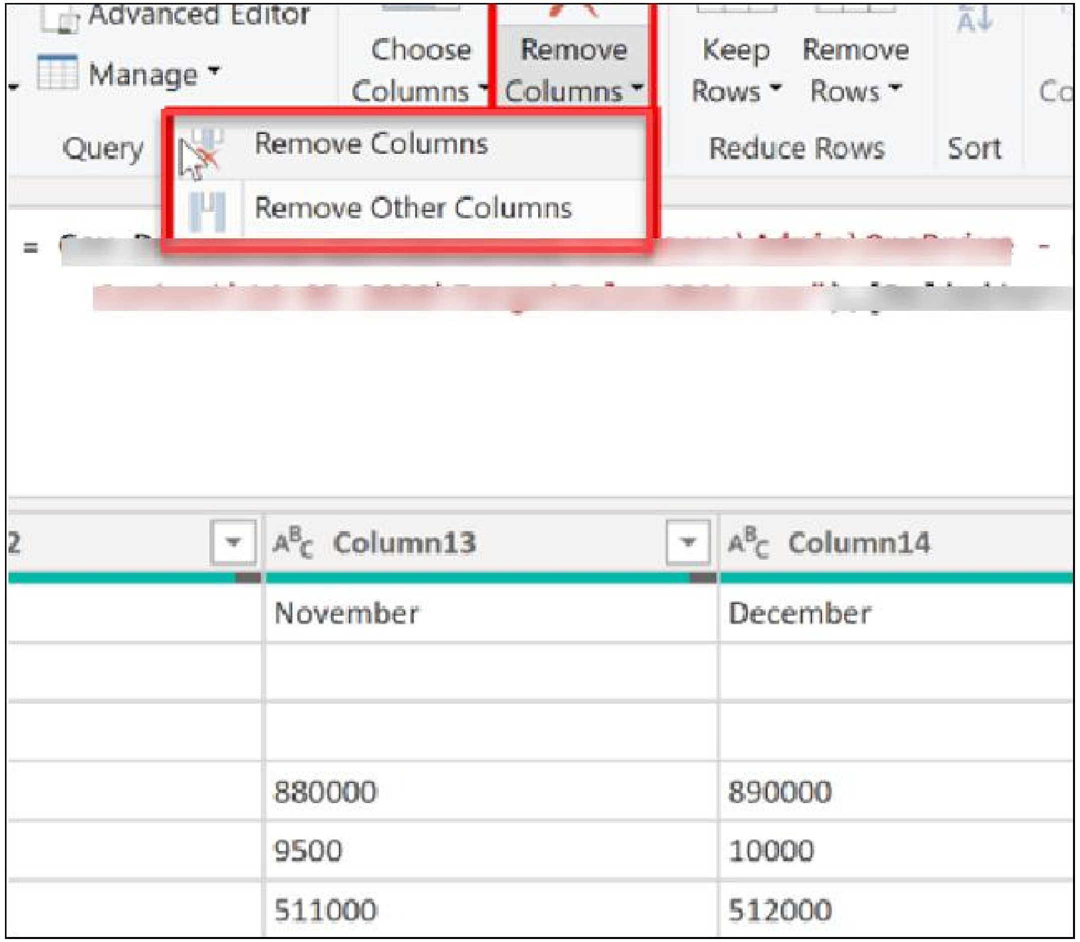



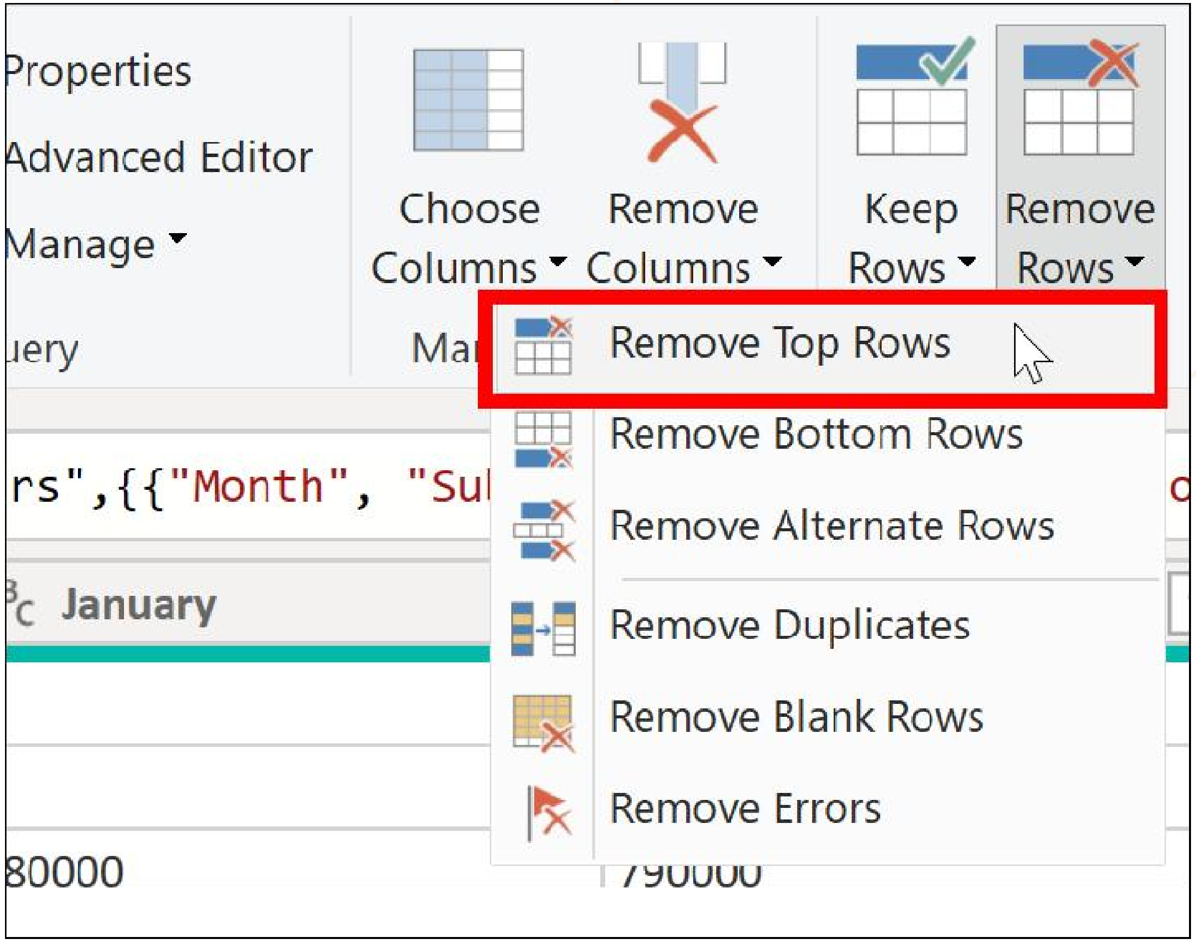
# Choose user-friendly values



# Shaping table structure



# Evaluate and change column data types

| SalesOrderNumber | OrderDate | TotalCost |
|---|---|---|
| SO43659 | 7/1/2017 | 2024.99 |
| SO43659 | 7/1/2017 | 6074.97 |
| SO43659 | 7/1/2017 | 2024.99 |
| SO43659 | 7/1/2017 | 2039.99 |
| SO43659 | 7/1/2017 | 2039.99 |
| SO43659 | 7/1/2017 | 4079.98 |
| SO43659 | 7/1/2017 | 2039.99 |
| SO43659 | 7/1/2017 | 86.52 |
| SO43659 | 7/1/2017 | 28.84 |
| SO43659 | 7/1/2017 | 34.2 |

Couldn't load the data for this visual

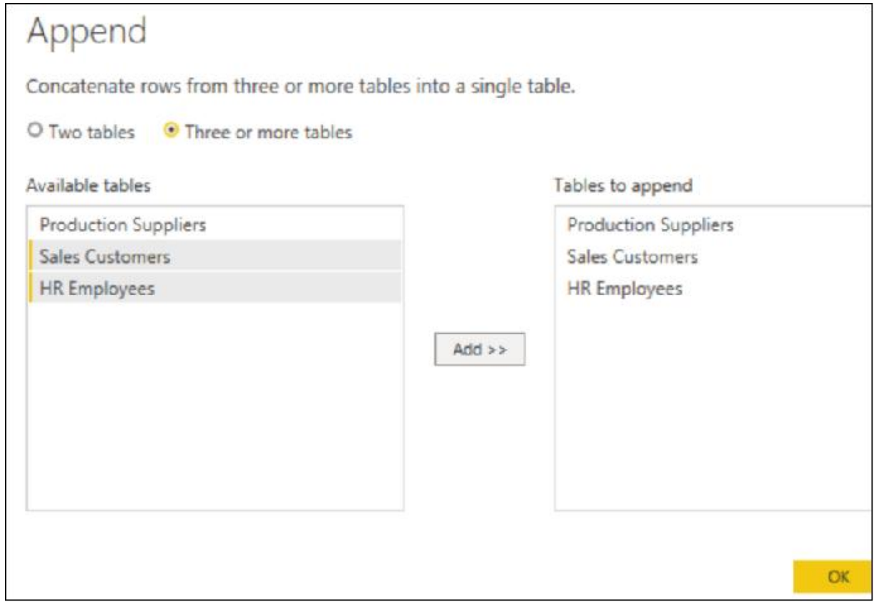
MdxScript(Model) (19, 40) Calculation error in measure 'Sales'[Quantity of Orders YTD]: A column specified in the call to function 'TOTALYTD' is not of type DATE. This is not supported.

Copy details

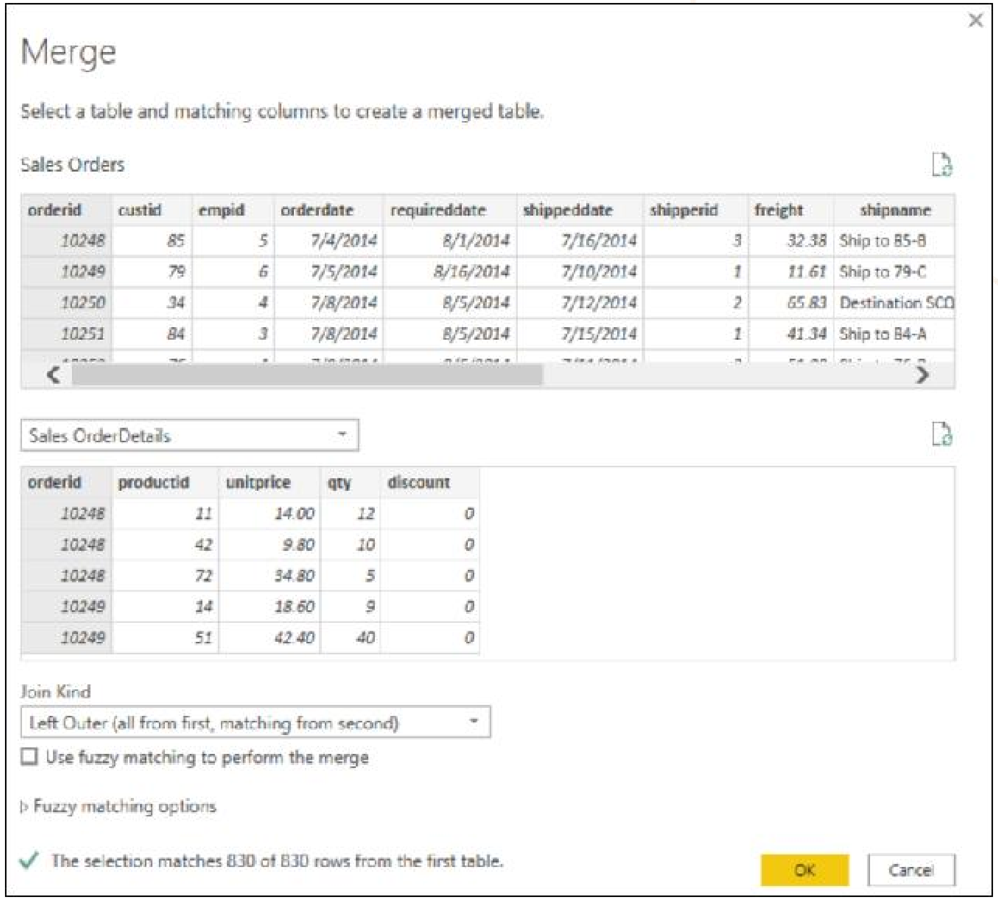
Send a Frown    Close

# Combine multiple queries into one

## Append




## Merge



# Unpivot or pivot columns

Add or remove table structure to meet your aggregation needs.

| | Category Name | Subcategory Name |
|---|---|---|
| 1 | Bikes | Mountain Bikes |
| 2 | Bikes | Road Bikes |
| 3 | Bikes | Touring Bikes |
| 4 | Clothing | Bib-Shorts |
| 5 | Clothing | Caps |
| 6 | Clothing | Gloves |
| 7 | Clothing | Jerseys |
| 8 | Clothing | Shorts |
| 9 | Clothing | Socks |
| 10 | Clothing | Tights |
| 11 | Clothing | Vests |
| 12 | Accessories | Bike Racks |
| 13 | Accessories | Bike Stands |
| 14 | Accessories | Bottles and Cages |

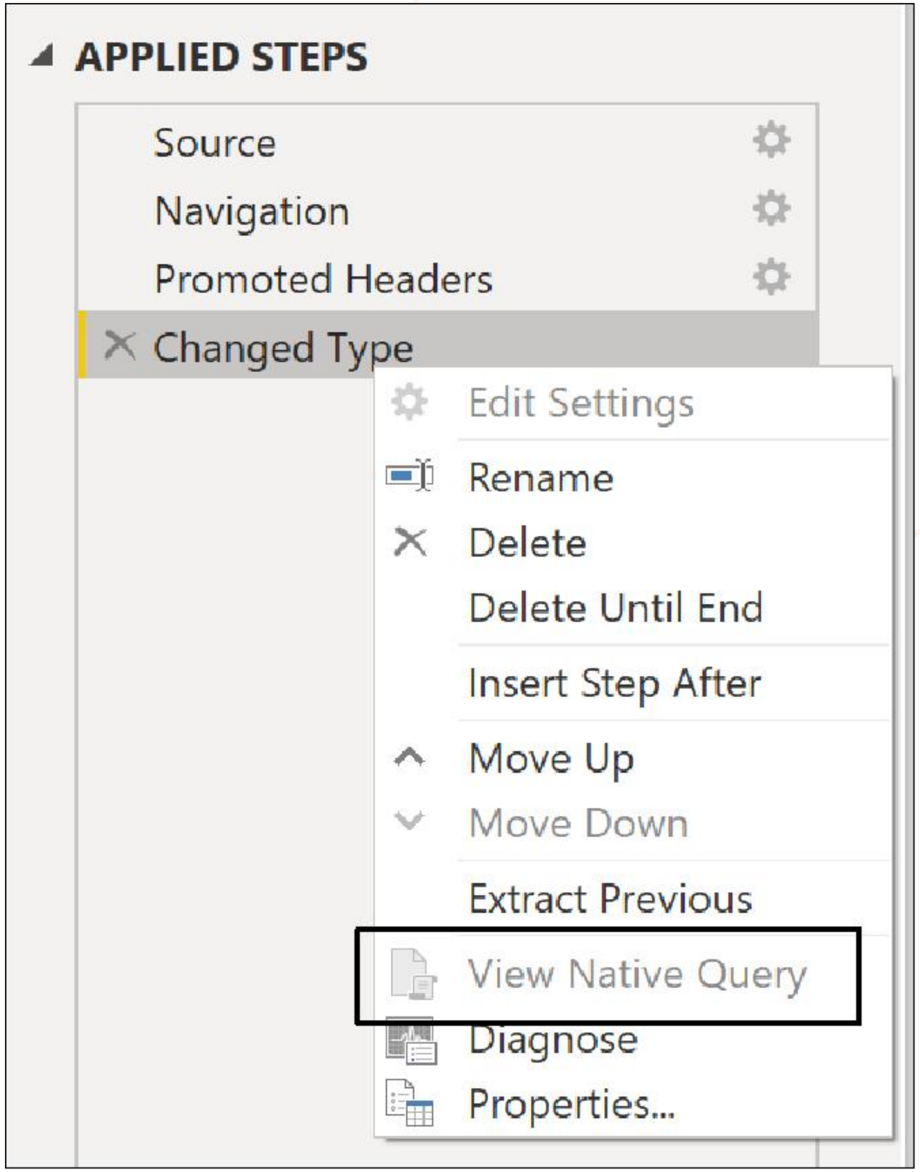
| | Bikes | Components | Clothing | Accessories |
|---|---|---|---|---|
| 1 | 3 | 14 | 8 | 12 |

# Query folding

Pushes data transformations to the source for better performance and efficiency.

Supported sources

- Relational databases
- OData feeds
- Active Directory



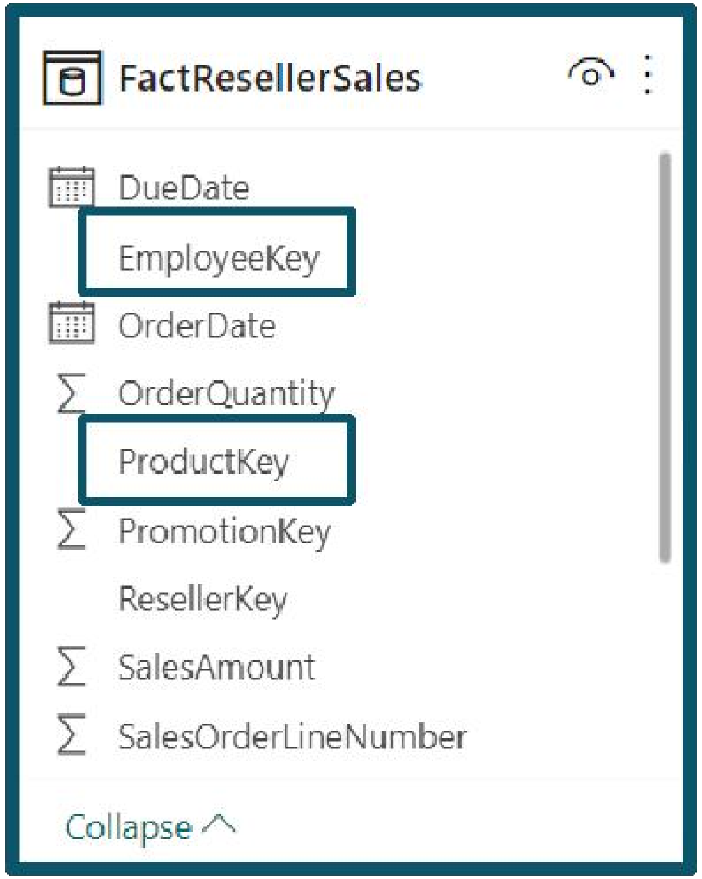
# Performance recommendations

- Only keep necessary data
- Check data types
- Reduce cardinality
- Disable query load
- Use parameters

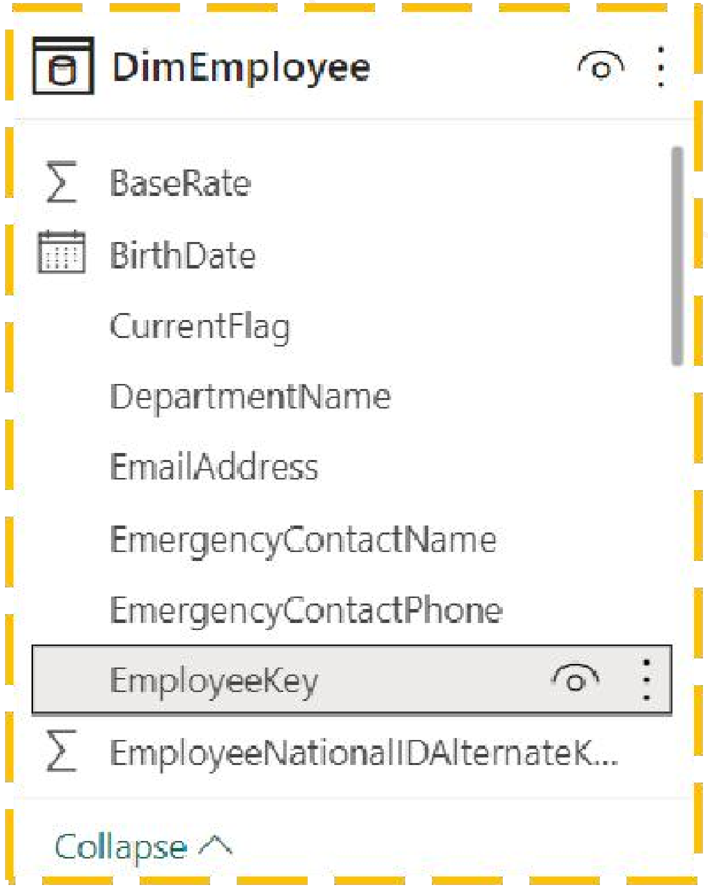
# Design a semantic model in Power BI

# Data table types

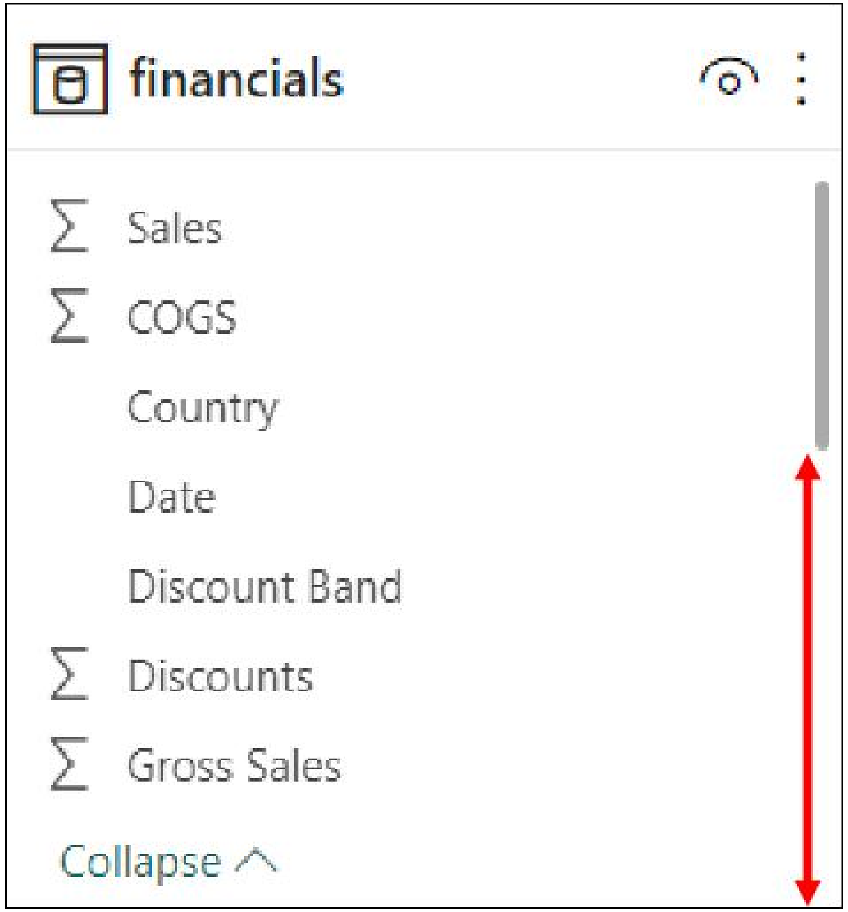
Fact tables are activities or events.

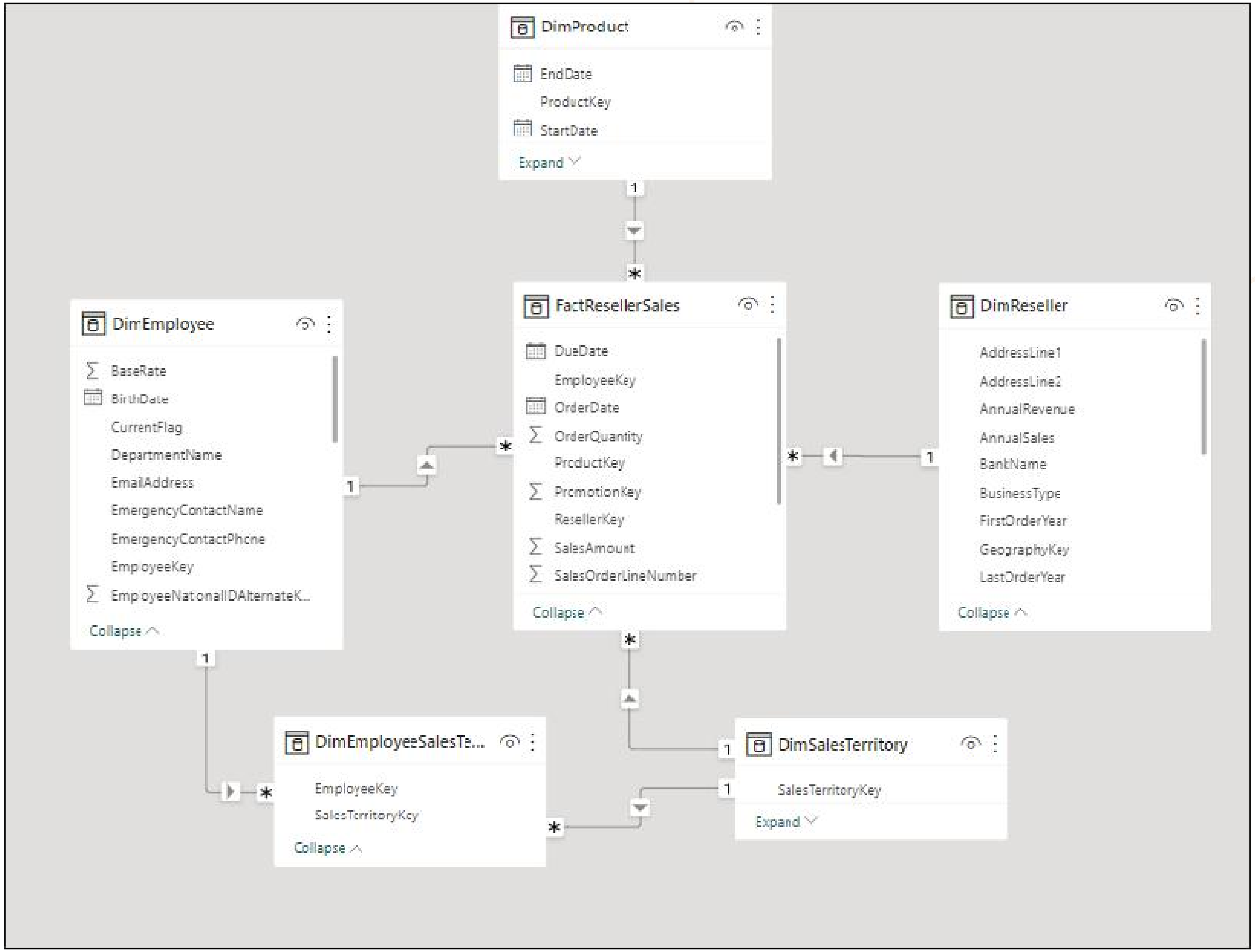



Dimension tables provide the details.

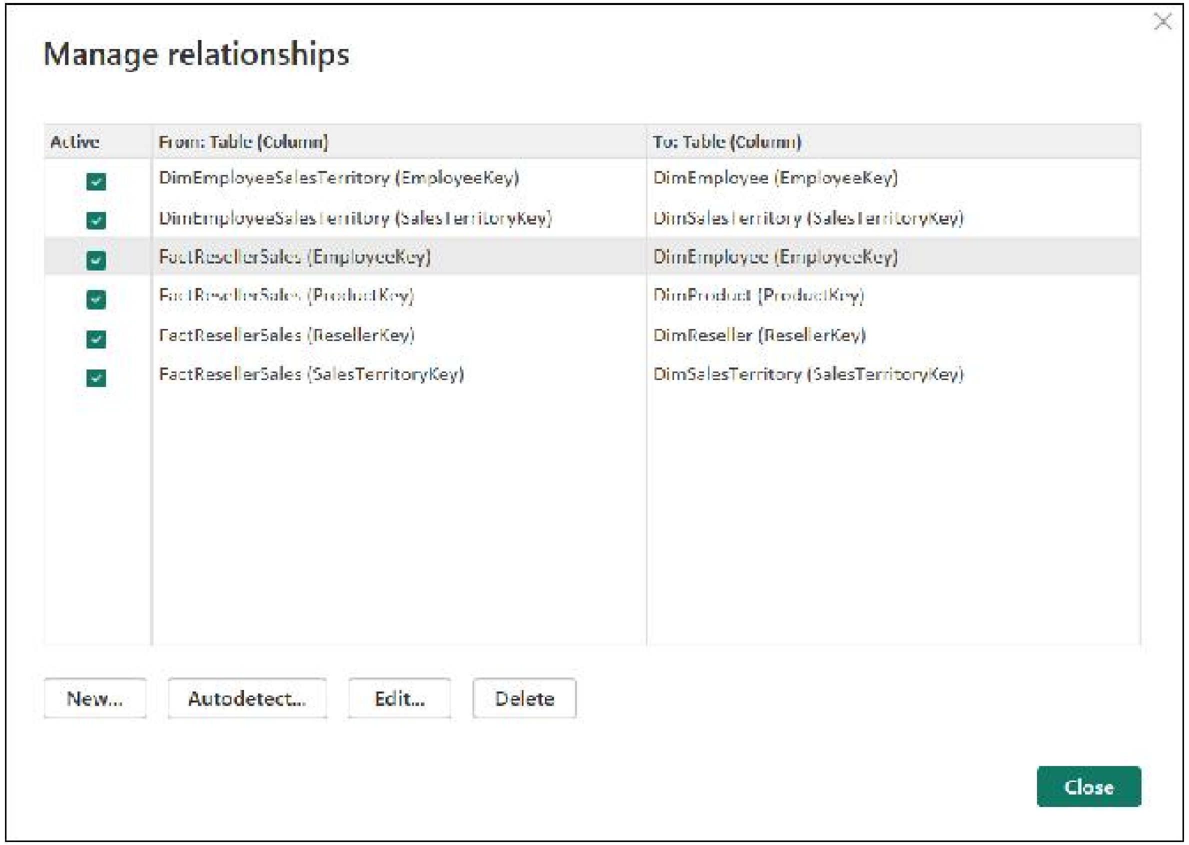


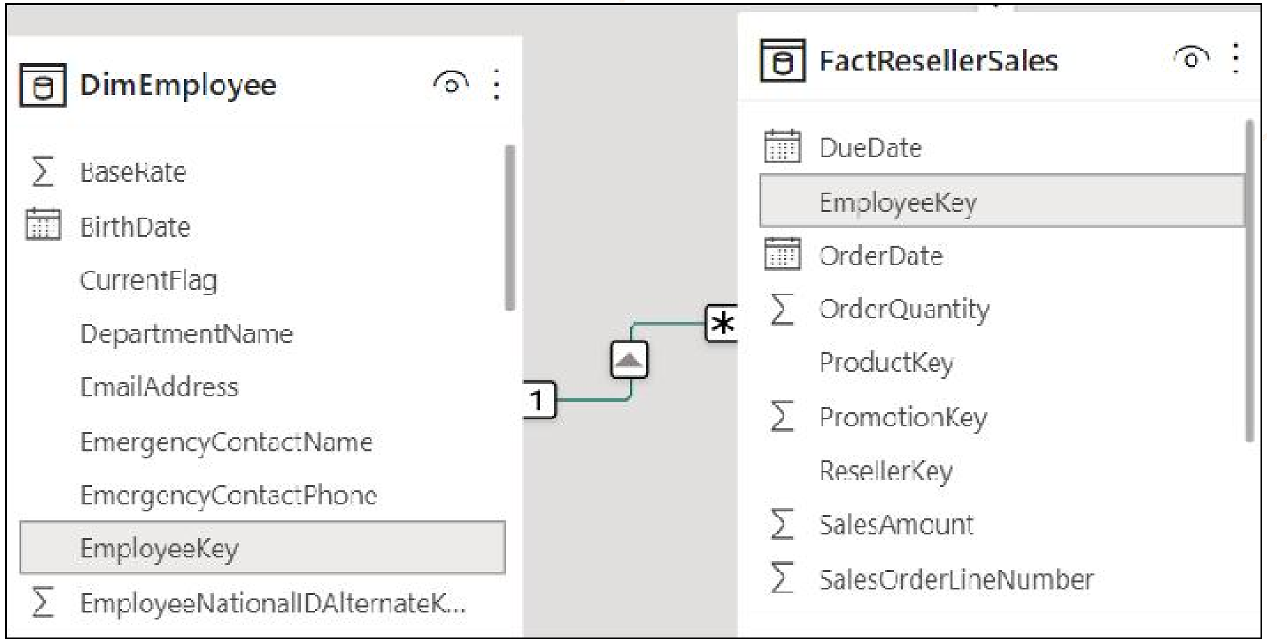
# Understand star schemas



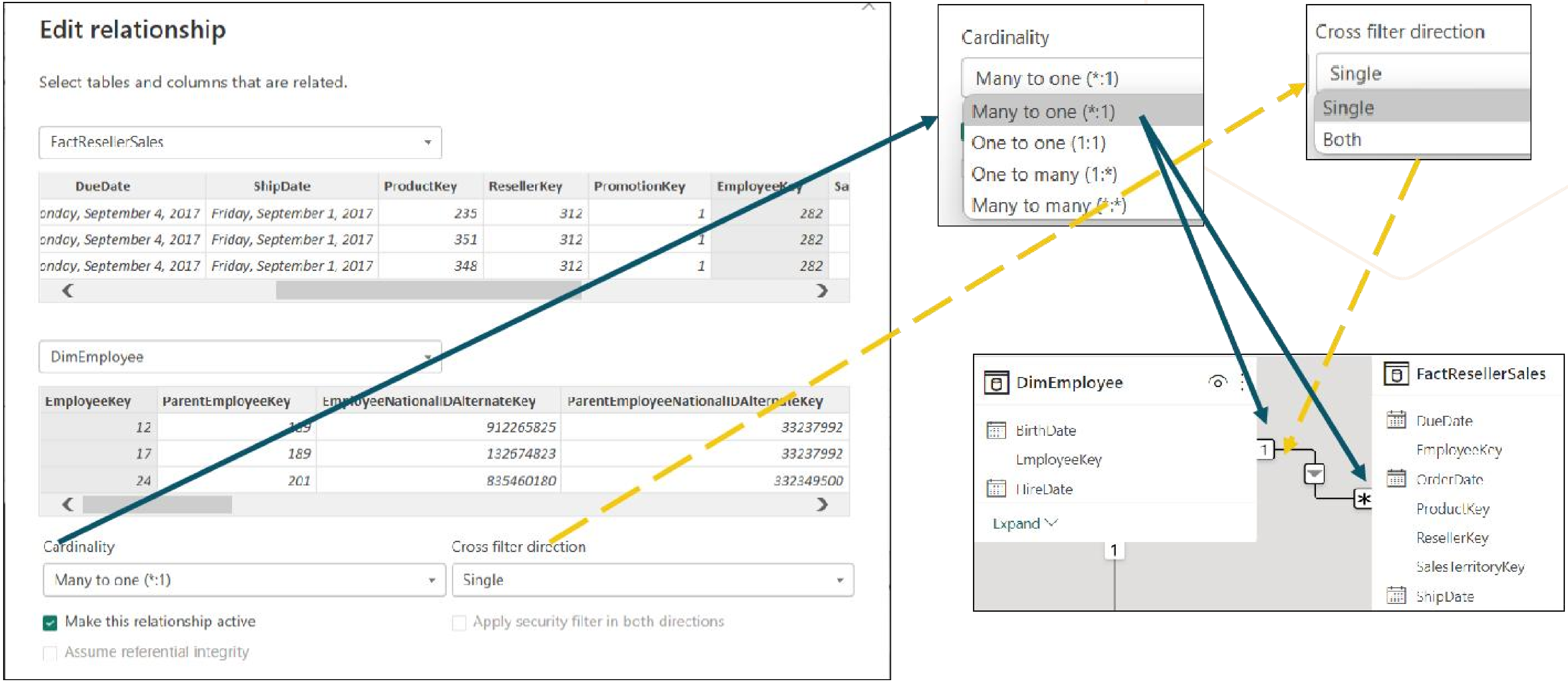



# Create relationships

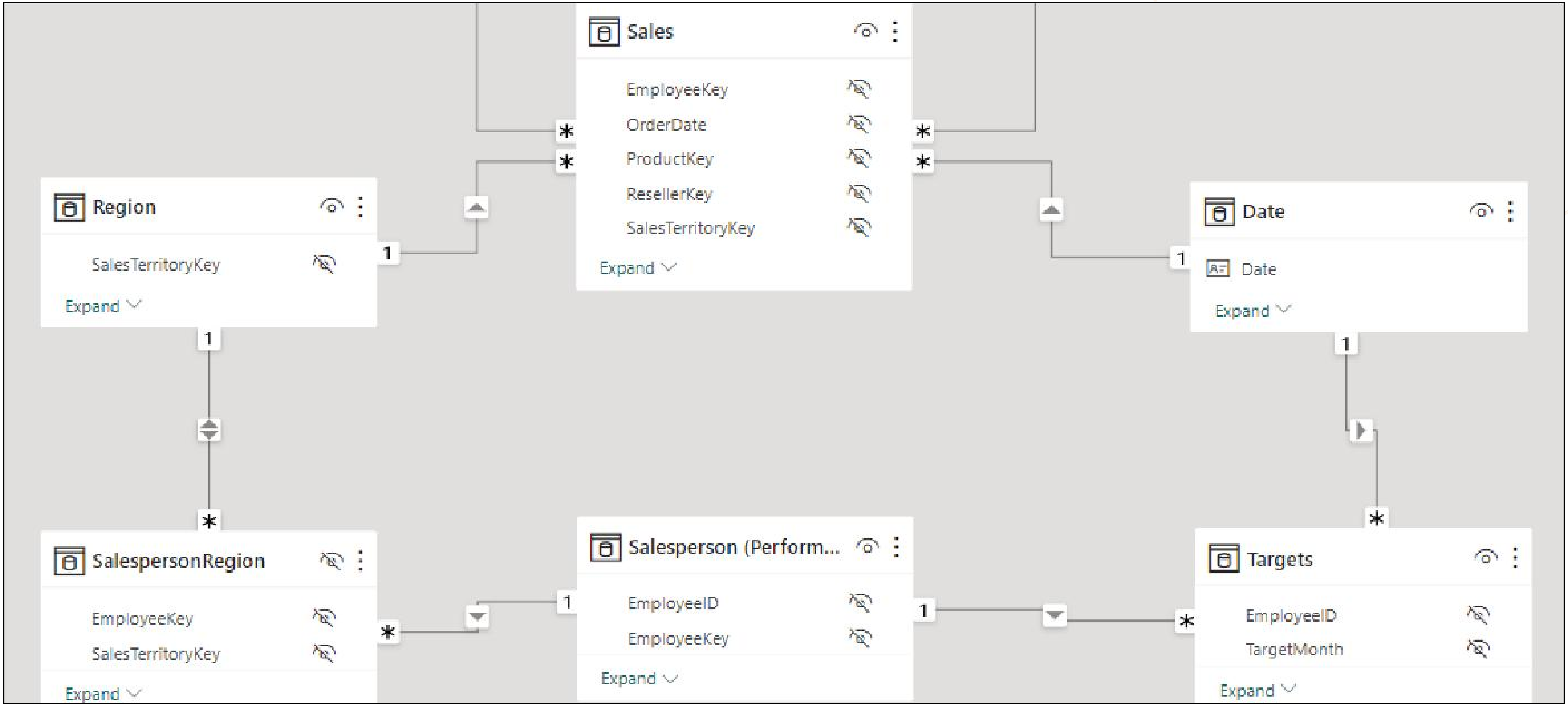


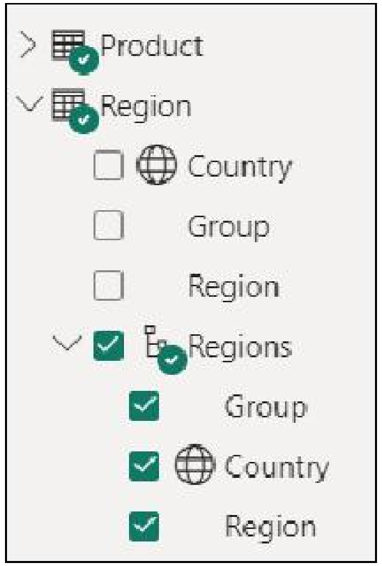
# Edit relationships

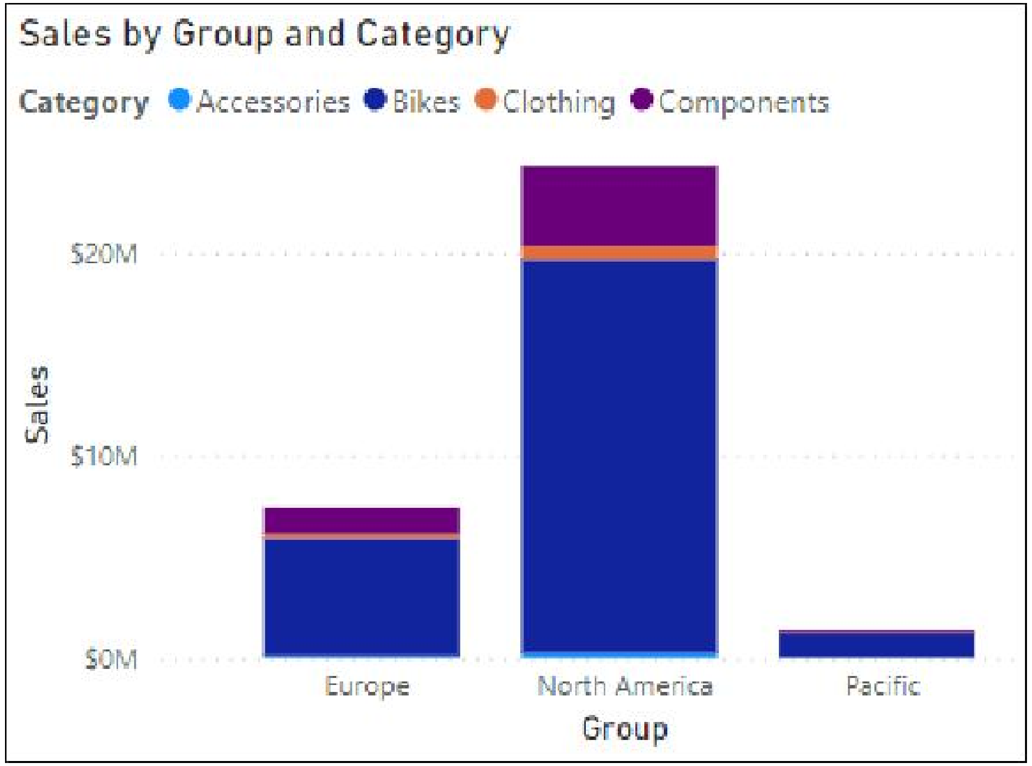


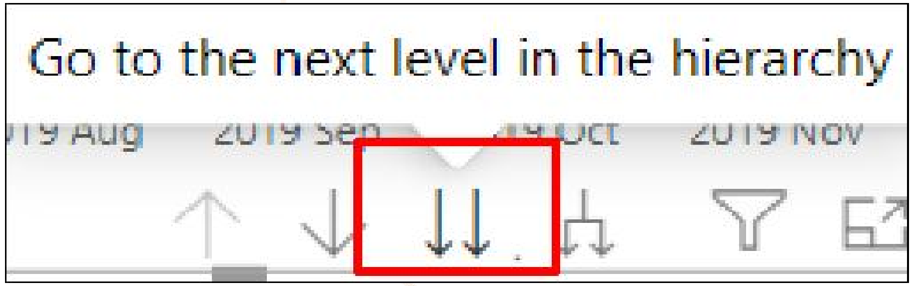
# Implications of circular relationships

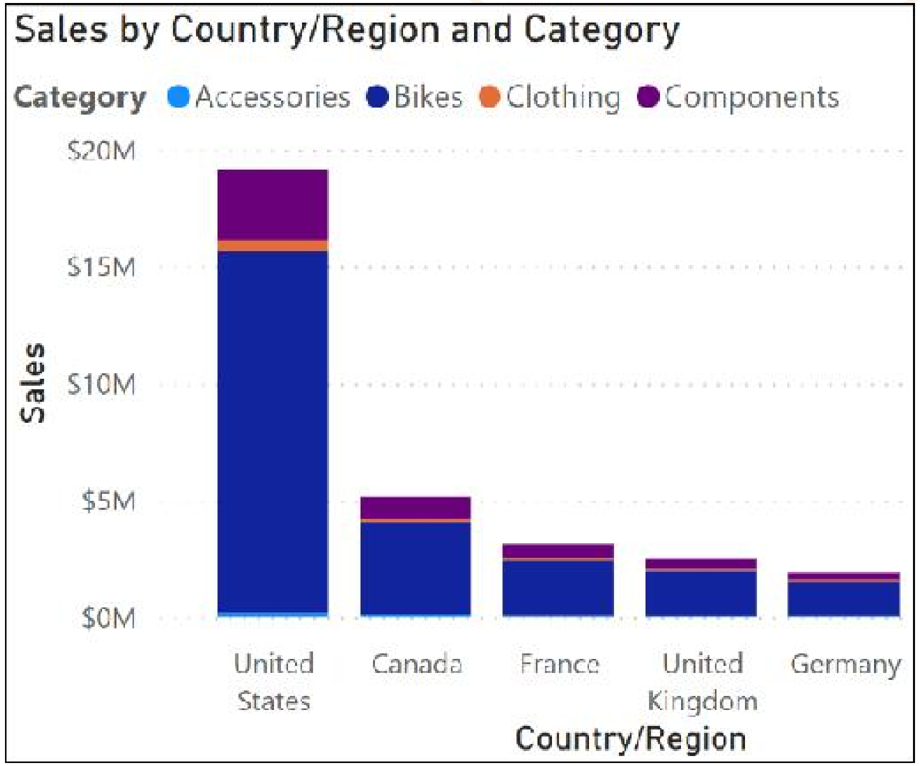


# How to use hierarchies for data fields



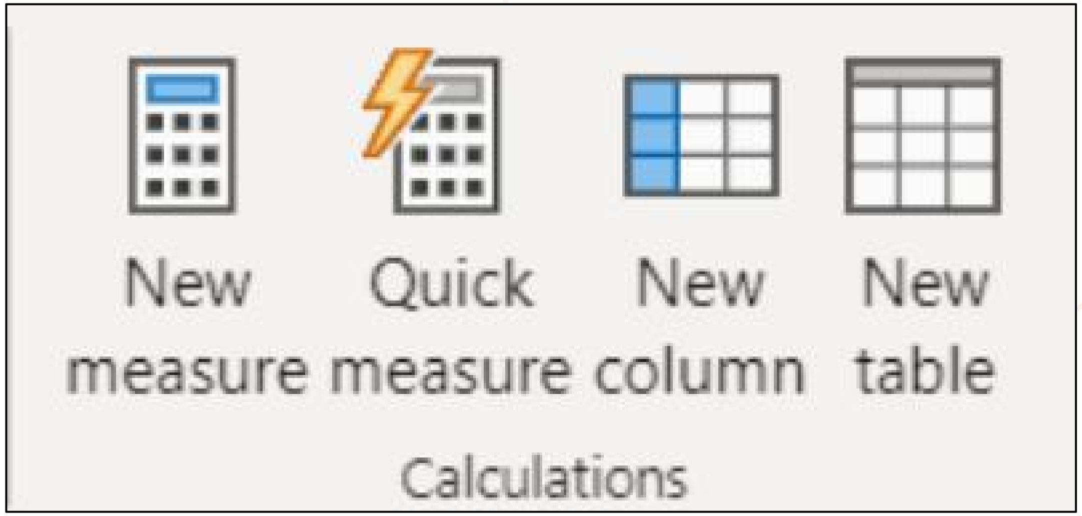



# Add measures to semantic models

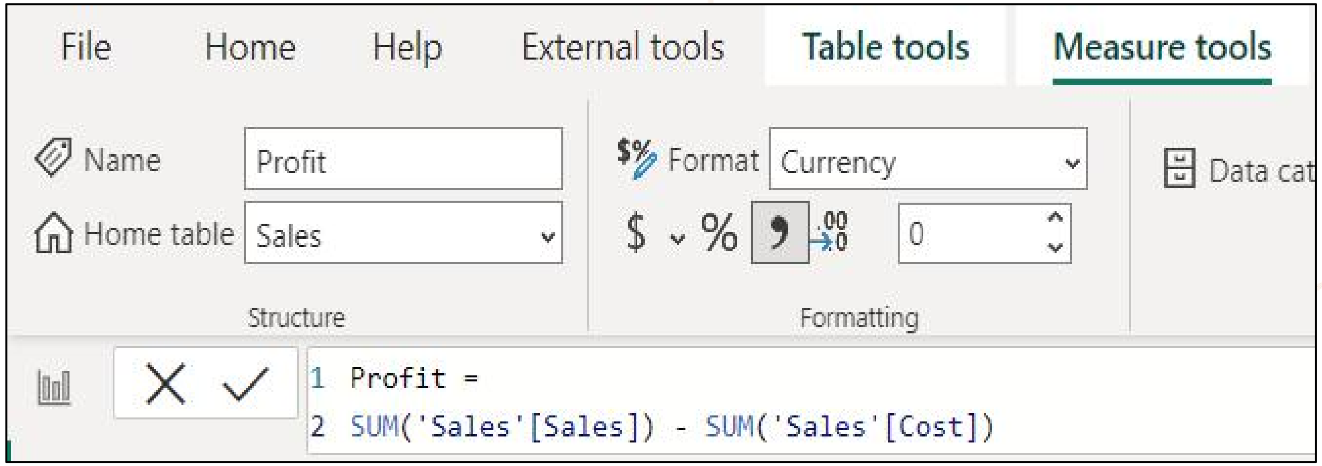
# What is DAX?

- **D**ata **A**nalysis E**x**pressions
- Library of functions and operators
- Build formulas and expressions
- Create calculated measures, columns, and tables

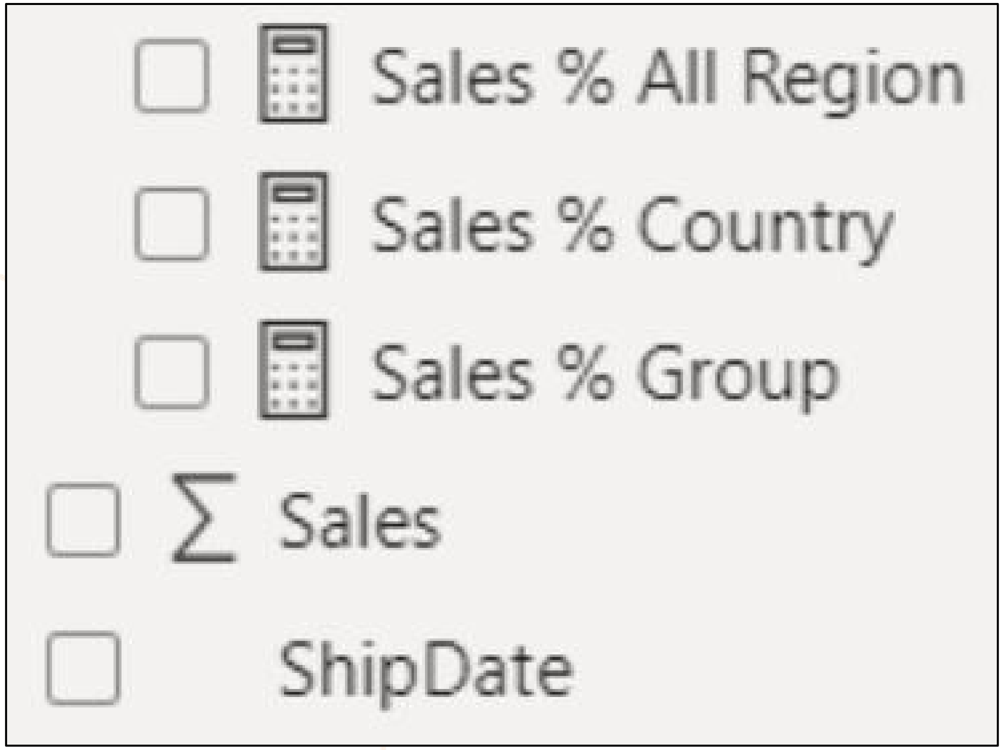


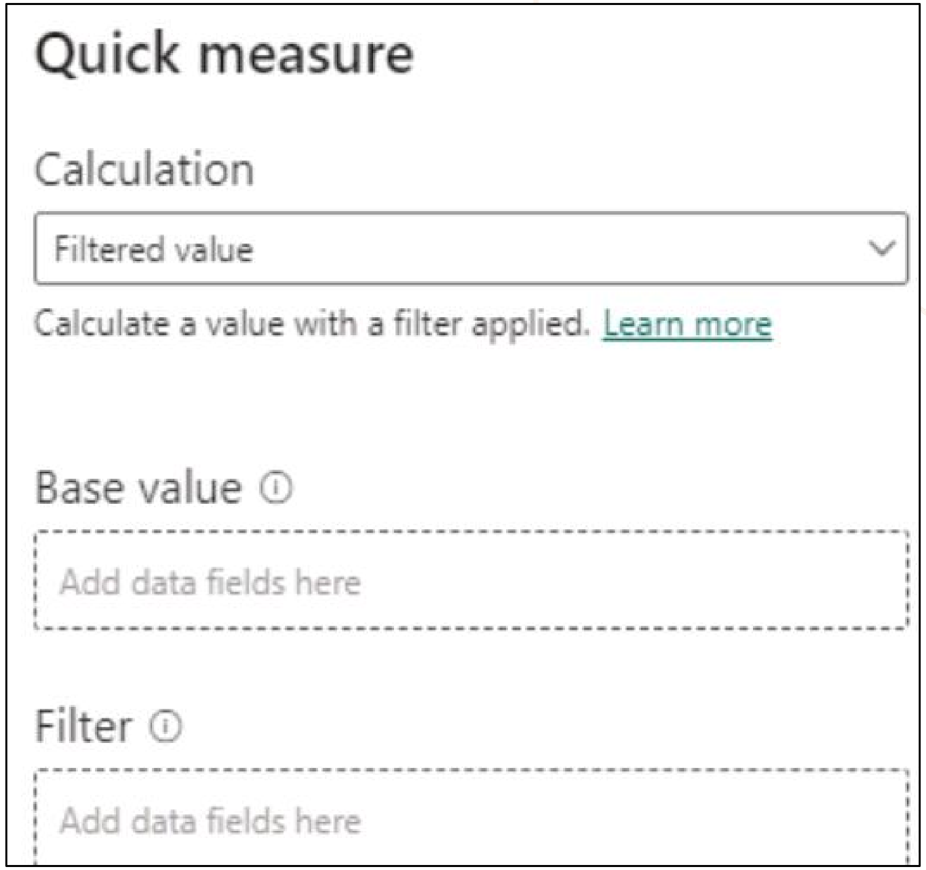
# Create calculated measures

- Defined with DAX definitions
- Computed on the fly.
- Not stored in data model.
- Responsive to interactions.
- Indicated by calculator icon.



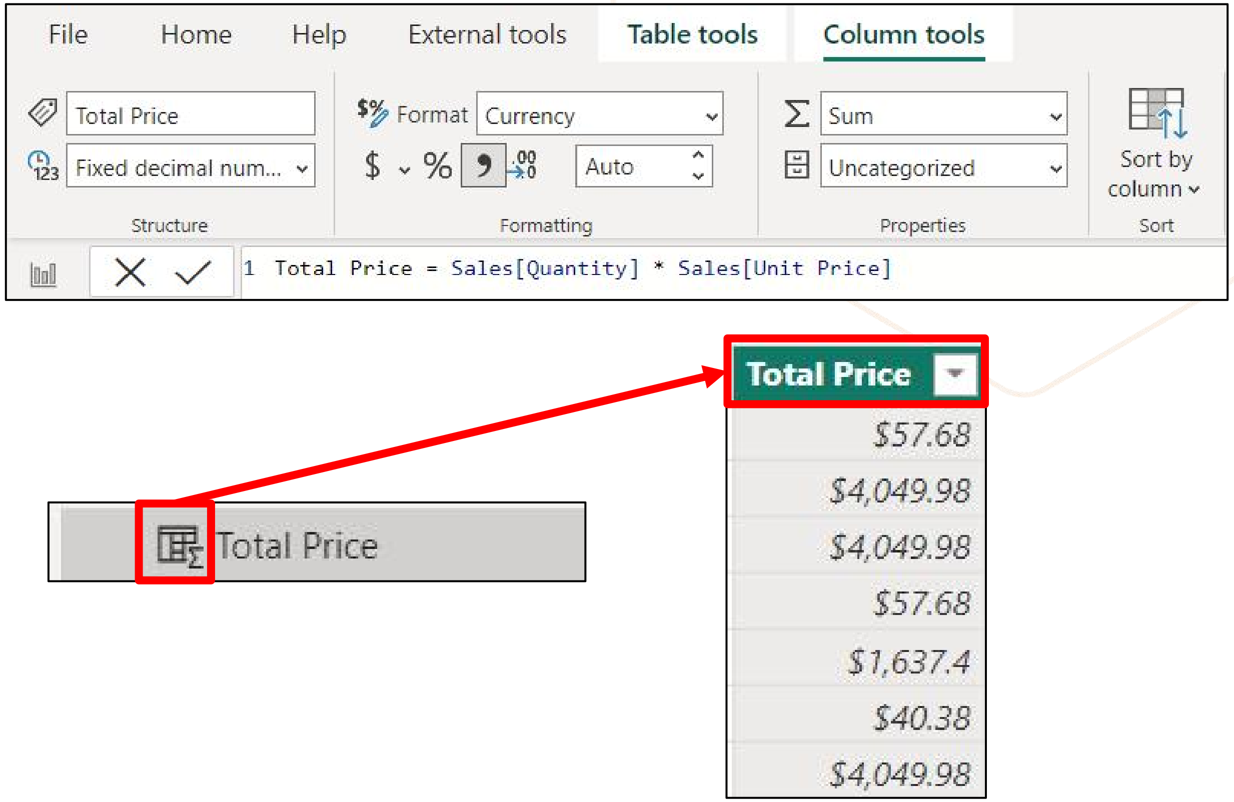

# Implicit vs. Explicit vs. Quick measures



# Create calculated columns

- Defined using DAX expressions.
- Computed & stored in data model.
- Useful “helper/connector columns.”
- Recalculated during data refresh.
- Table and Sigma icon.



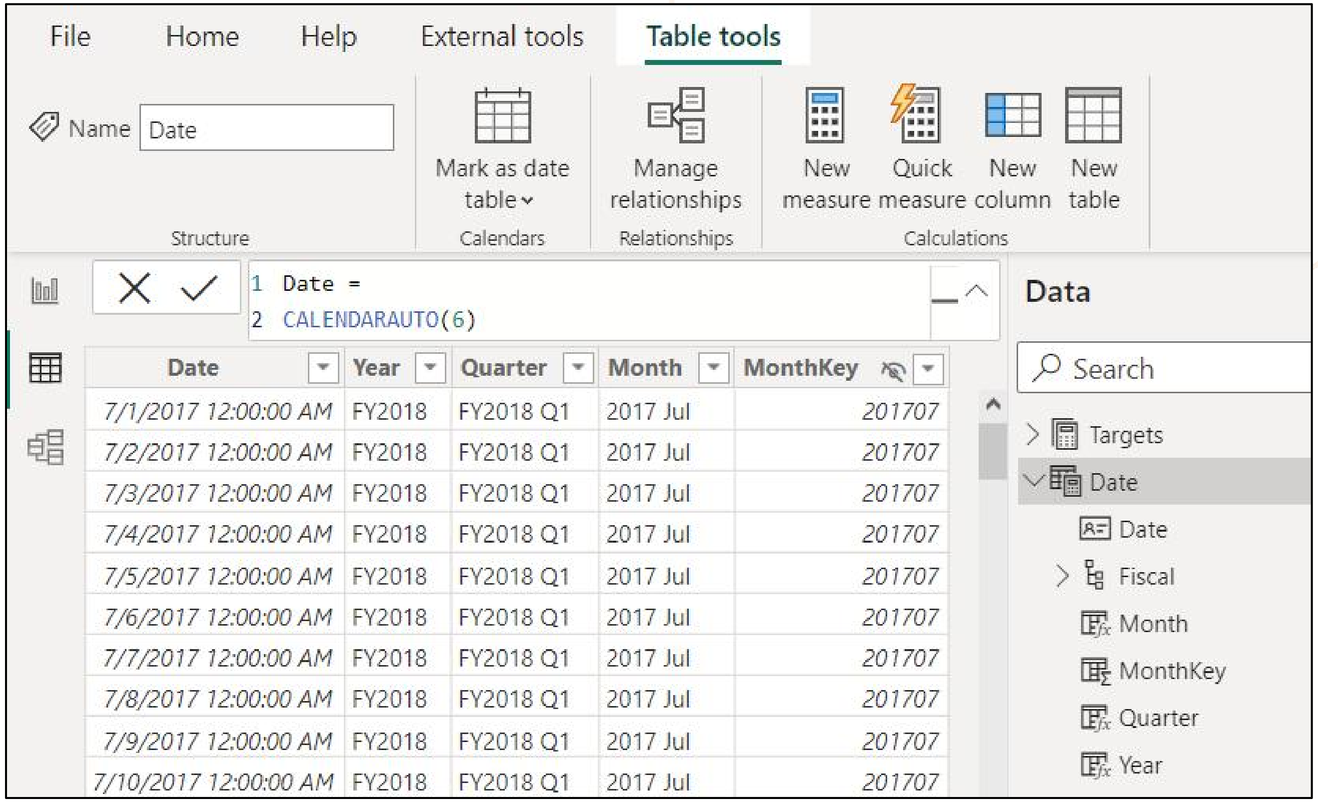
# Columns vs. measures

- Calculated columns:
  - Create values for each row in table.
  - Store values in the .pbix file.
  - Increases data model size.
  - Row-by-row calculation can impact performance.
  - Must be referenced with measures for reuse.

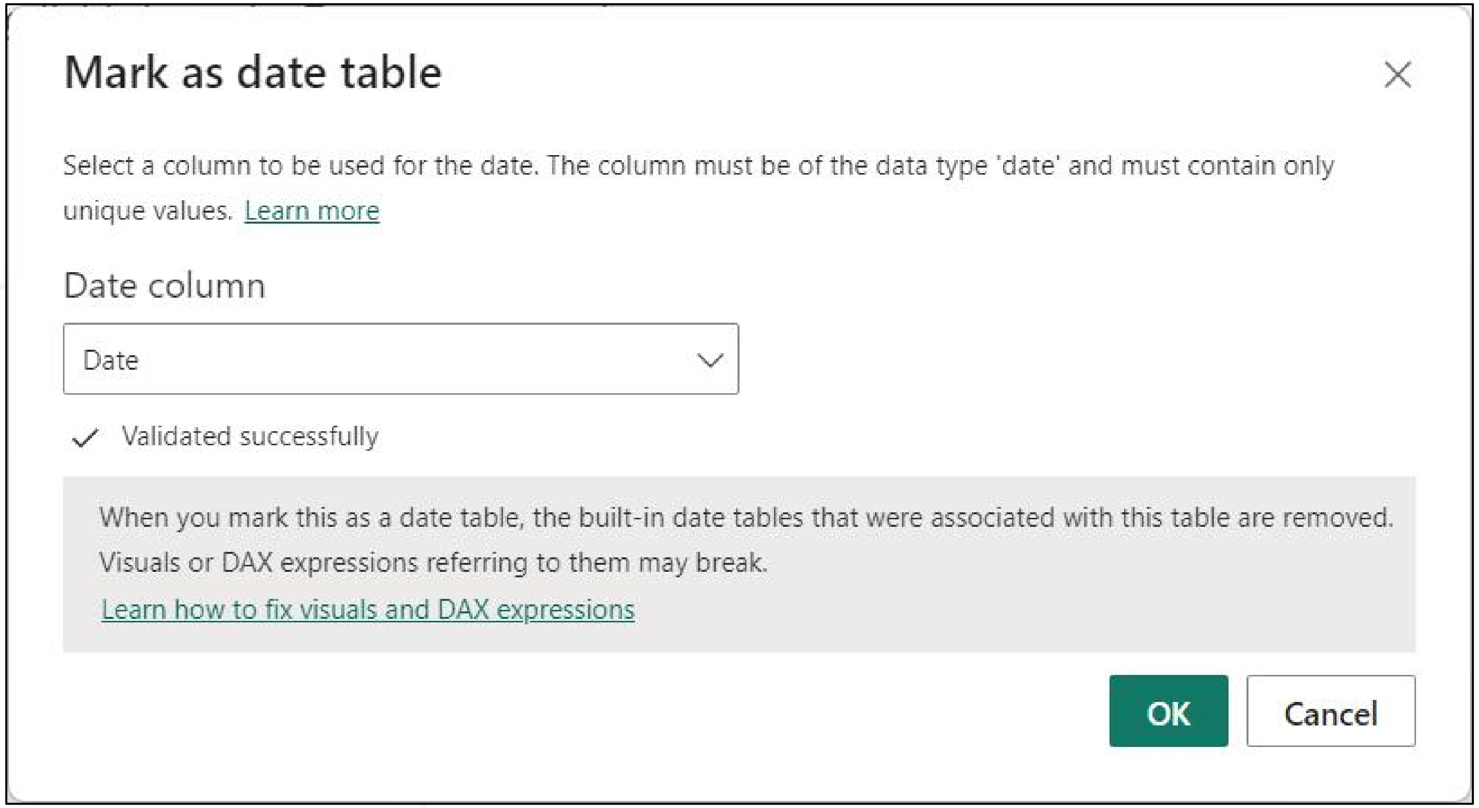
- Measures:
  - Calculate on demand.
  - Calculated based on filters.
  - Doesn’t affect data model size.
  - DAX expressions may still be suboptimal.
  - Can reference other measures directly for reuse.

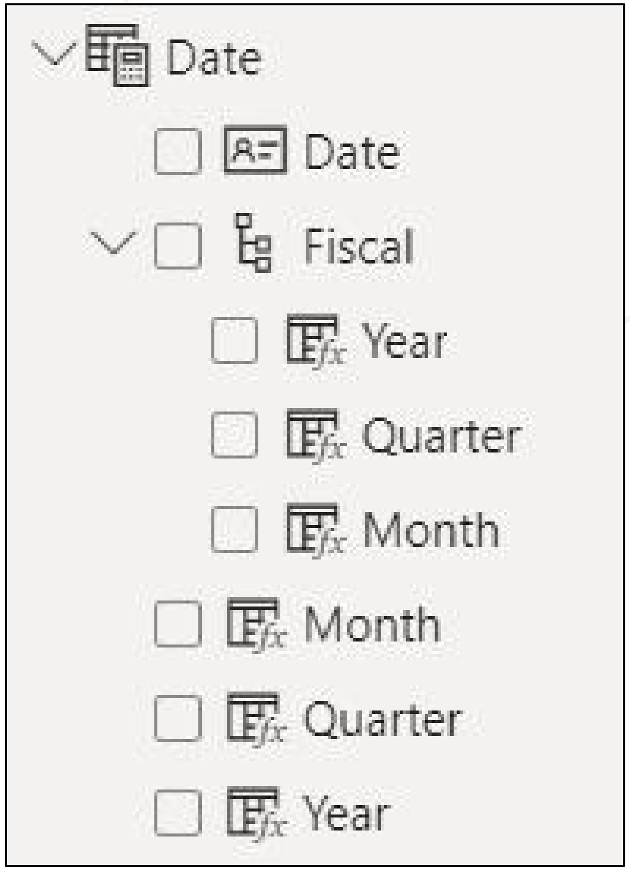
# Create calculated tables

- Defined using DAX expressions.
- Computed & stored in data model.
- Useful for aggregating data or creating custom tables.
- Table and calculator icon.



# Create a common date table

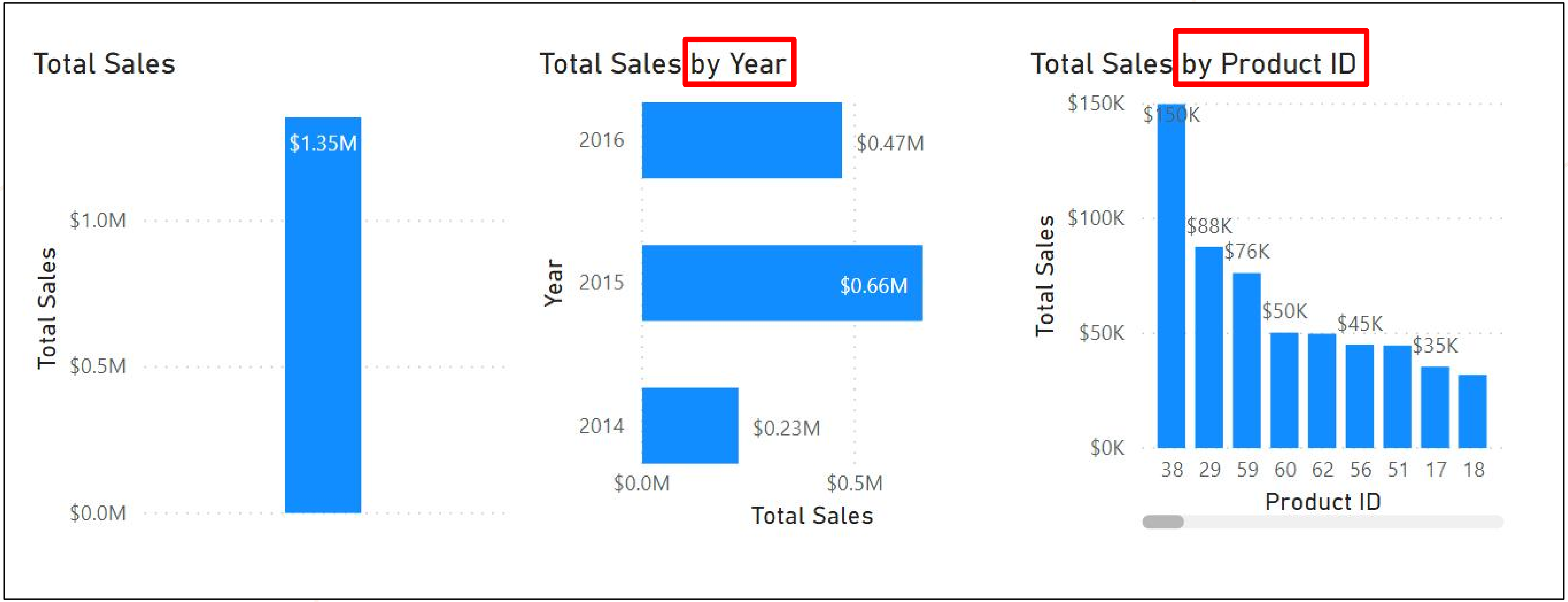


# Use DAX functions

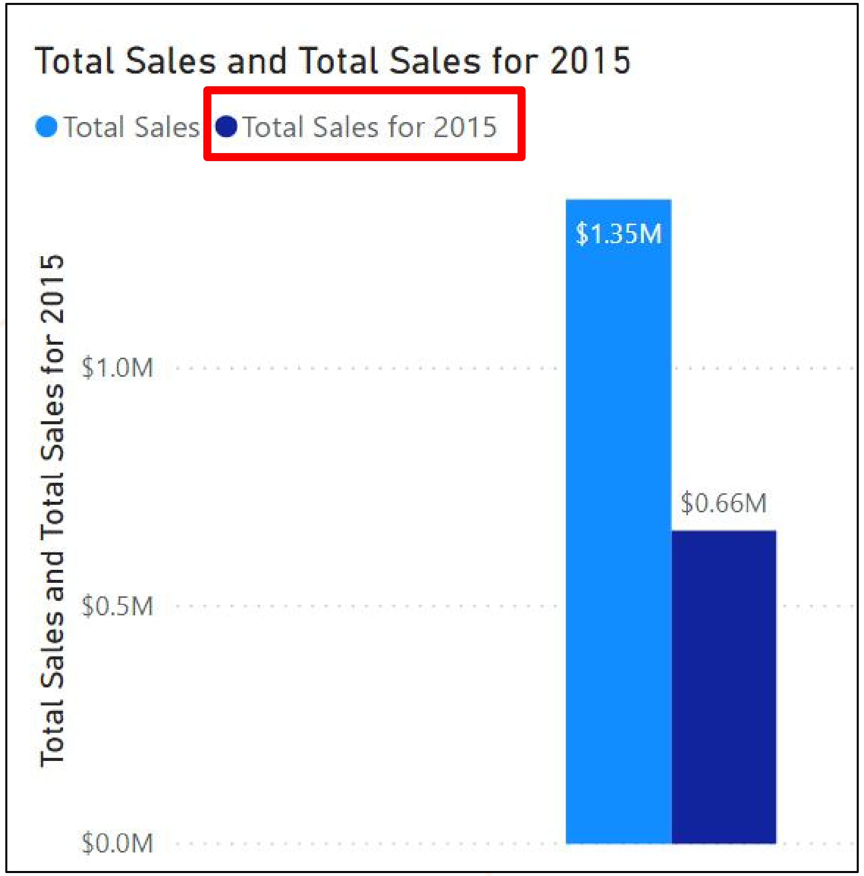
# Understanding filter context

- Measures are contextually different, or “dynamic,” depending on filters.



# CALCULATE() function

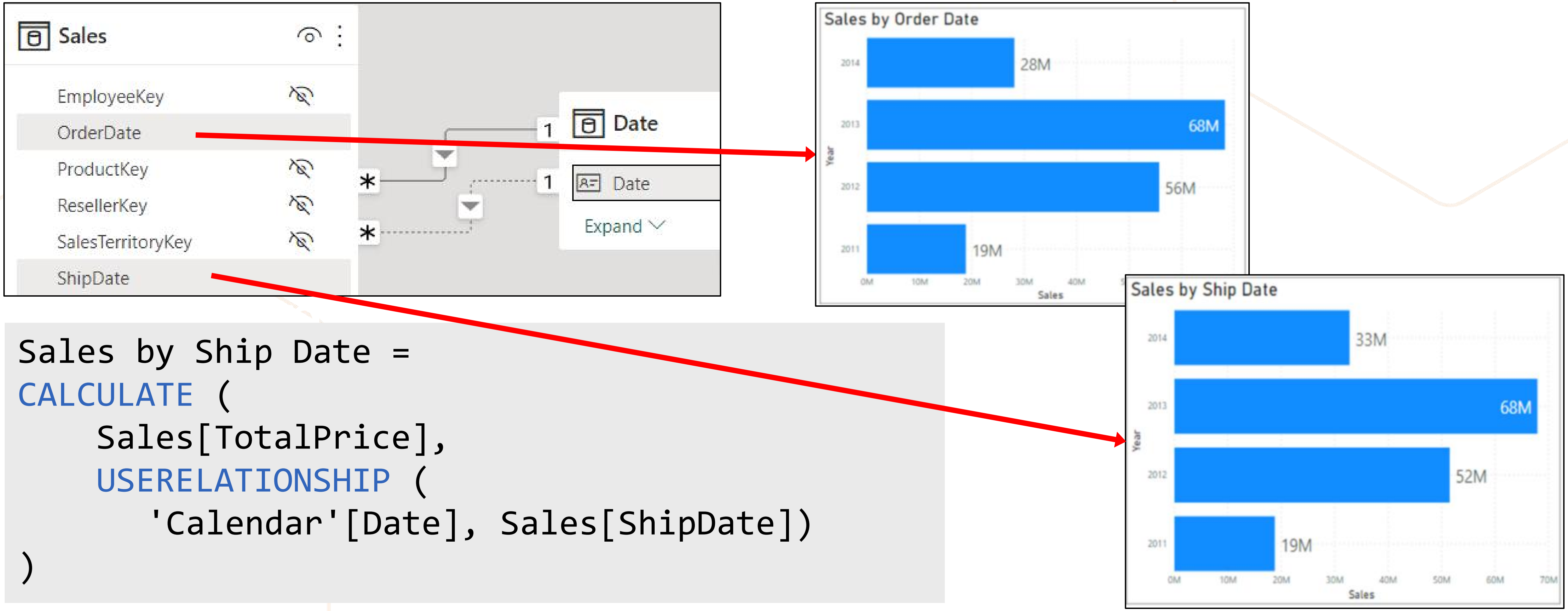
- Adjusts how measures interpret data filters, enabling context control.




```
Total Sales for 2015 =
CALCULATE (
    SUM ( 'Sales OrderDetails'[Total
Price] ),
    YEAR ( 'Sales OrderDetails’[OrderDate] )
= 2015
)
```

# Use inactive relationships with DAX

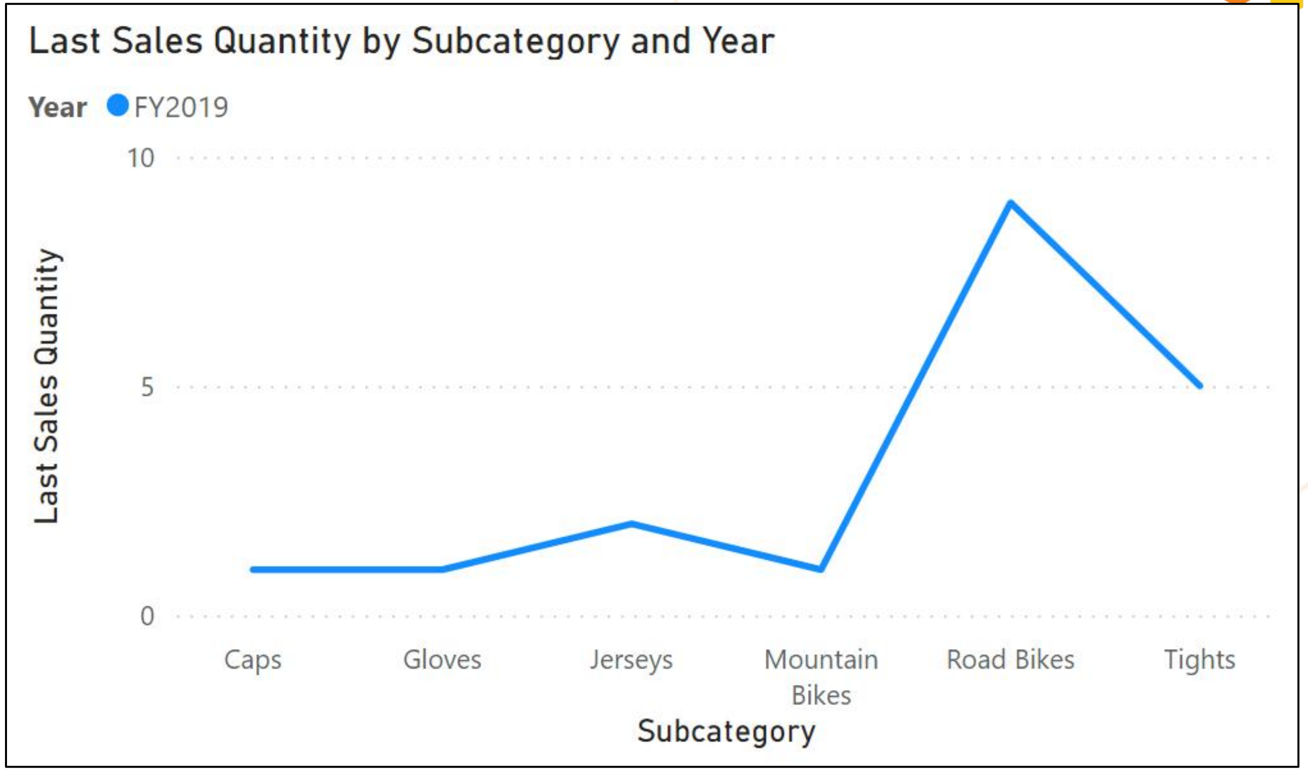
- Enable additional table filtering without impacting the active relationship.




```
Sales by Ship Date =
CALCULATE (
    Sales[TotalPrice],
    USERELATIONSHIP (
        'Calendar'[Date], Sales[ShipDate])
)
```

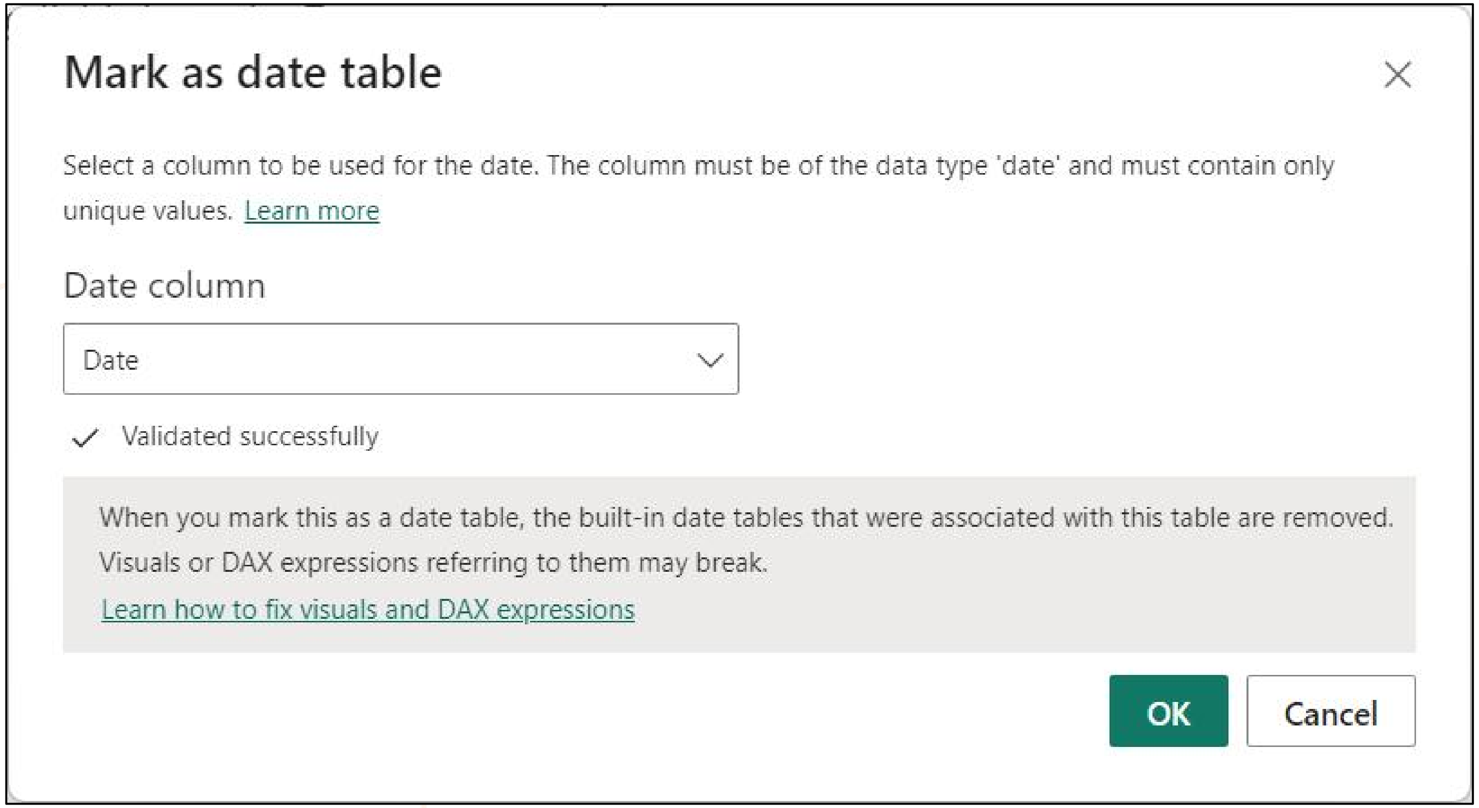
# Semi-additive measures

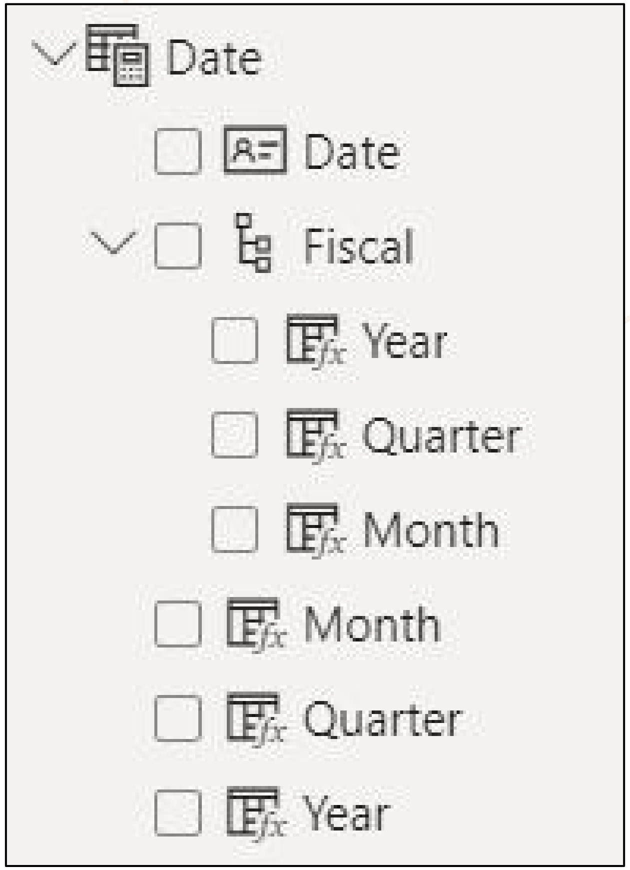
- **Context-dependent**: Results vary based on dimensions.
- **Time Intelligence**: Useful for averages, ratios, balances.
- **Caution with aggregation**: Beware of misleading results when aggregating.




```
Last Sales Quantity =
CALCULATE(
    SUM(Sales[Quantity]),
      LASTDATE('Date'[Date])
)
```

# Create a common date table



# Time intelligence functions

| Month | 2014 | 2015 | 2016 |
|---|---|---|---|
| January | | $66,692.8 | $100,854.72 |
| February | | $107,900 | $205,416.67 |
| March | | $147,879.9 | $315,242.12 |
| April | | $203,579.29 | $449,872.68 |
| May | | $260,402.99 | $469,771.34 |
| June | | $299,490.99 | $469,771.34 |
| July | $30,192.1 | $354,955.92 | $469,771.34 |
| August | $56,801.5 | $404,937.61 | $469,771.34 |
| September | $84,437.5 | $464,670.63 | $469,771.34 |
| October | $125,641.1 | $534,999.13 | $469,771.34 |
| November | $175,345.1 | $580,912.49 | $469,771.34 |
| December | $226,298.5 | $658,388.75 | $469,771.34 |
| **Total** | **$226,298.5** | **$658,388.75** | **$469,771.34** |

```
Total Sales Previous Month =
CALCULATE ( SUM ( Sales[Total Price] ),
PREVIOUSMONTH ( 'Date'[Date] ) )
```

| Year | Month | Total Sales | Total Sales Previous Month |
|---|---|---|---|
| 2015 | March | $39,979.9 | $41,207.2 |
| 2015 | April | $55,699.39 | $39,979.9 |
| 2015 | May | $56,823.7 | $55,699.39 |
| 2015 | June | $39,088 | $56,823.7 |
| 2015 | July | $55,464.93 | $39,088 |
| 2015 | August | $49,981.69 | $55,464.93 |
| 2015 | September | $59,733.02 | $49,981.69 |
| 2015 | October | $70,328.5 | $59,733.02 |
| 2015 | November | $45,913.36 | $70,328.5 |
| 2015 | December | $77,476.26 | $45,913.36 |

# Use variables to improve performance and readability

**Without variable:**

```
Sales YoY Growth =
   DIVIDE (
       ( [Sales] - CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) ) ),
       CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) ) )
```

**With variable:**

```
Sales YoY Growth =
     VAR SalesPriorYear = CALCULATE ( [Sales], PARALLELPERIOD ( 'Date'[Date], -12, MONTH ) )
     VAR SalesVariance =  DIVIDE ( ( [Sales] - SalesPriorYear ), SalesPriorYear )
  RETURN
  SalesVariance
```

# Thank you
# Let's discuss our collaborations

PT Nawa Darsana Teknologi
Gedung Office 8, 18th Floor, Jl. Jend Sudirman Kav 52-53
SCBD Lot 8, Senopati Jakarta Selatan 12190 DKI Jakarta, Indonesia
Website : www.nawatech.co
Phone : +62 21 29552754



**Contact**
-
-
**Email** :
**Phone** :